

How to kickstart $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ with Short Authentication Strings and Out-Of-Band Communication

Wasilij Beskorovajnov³ and Jörn Müller-Quade^{1,2,3}

¹ Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany

² KASTEL Security Research Labs, Karlsruhe, Germany
mueller-quade@kit.edu

³ FZI Research Center for Information Technology, Karlsruhe, Germany
Beskorovajnov@fzi.de

Abstract. We study “send this data to that device now” exchanges under an active network adversary without PKI or pre-shared secrets. We model a human-verifiable out-of-band channel for comparing short codes as a first-class UC functionality $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$. Building on this, we give two commitment-style protocols, π_{SAS} (one-sided) and $\pi_{\text{SAS}}^{\times}$ (mutual), that are direct UC equivalents of MANA IV and prove they realize $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ and $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ with explicit misbinding parameter $\varepsilon = 2^{-t}$. We then compose SAS-based authentication with standard KEM/DEM encryption to obtain a UC-secure message-transfer functionality $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$, preserving the explicit ε under composition, and we detail practical instantiations over signatures or MACs. Complementing the theory, we systematize real-world tooling: popular file-transfer utilities either form unauthenticated WebRTC/DTLS channels or use PAKE “one-code” designs that couple rendezvous and a long shared secret, but none deploy a session-bound SAS. Our approach decouples rendezvous from authentication and reduces the out-of-band burden to comparing a short t -bit string. We also sketch an RO-free variant (coin-flip plus non-malleable commitments) with the same user interface.

1 Introduction

There is a small but active ecosystem of “send this to that device now” tools: PAKE/one-code utilities such as Magic Wormhole⁴, WebWormhole⁵, and croc⁶; browser-based WebRTC/DTLS tools like PairDrop⁷ and FilePizza⁸. Despite different plumbing, all must answer the same question: is this the intended peer under an active network adversary?

⁴ <https://github.com/magic-wormhole/magic-wormhole>

⁵ <https://github.com/saljam/webwormhole>

⁶ <https://github.com/schollz/croc>

⁷ <https://github.com/schlagmichdoch/PairDrop>

⁸ <https://github.com/kern/filepizza>

A first observation is that today’s file-transfer tools do not deploy a session-bound, human comparison step. Instead we see two dominant patterns. (i) Unauthenticated DTLS over WebRTC: the browser checks a DTLS certificate against a fingerprint delivered by signaling, but there is no user-verifiable bind, so a malicious signaling service can splice two DTLS associations and sit in the middle; (ii) PAKE “one-code” designs: a single human-readable code carries both a rendezvous reference and a PAKE password. This yields smooth UX but requires moving a comparatively long codes out of band and shifts security to the secrecy and integrity of that code.

We revisit a different, well-studied bind: the *short authentication string* (SAS). Here each endpoint derives a short t -bit value from the live run (e.g., via a commitment-style transcript hash or a coin-flip bound to the transcript) and the user compares the two values over an auxiliary authenticated channel. Vaudenay showed that when users compare correctly, an active man-in-the-middle succeeds with probability at most 2^{-t} [Vau05]. This paper treats that human-verifiable step as a first-class component, models it in UC, and shows how SAS-based authentication composes to secure message transfer while keeping the out-of-band burden to comparing only a short t -bit string.

Contributions

1. **Ideal OOB/SAS functionality.** We introduce $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$, a UC ideal functionality for a hybrid, human-verifiable out-of-band channel that authenticates short strings. The current version handles synchronous comparisons.
2. **PISAS and PISAS[×] as UC equivalents of MANA IV.** We give protocols π_{SAS} (one-sided) and π_{SAS}^X (mutual) that follow the classical MANA IV commit–challenge–open mechanics [LP09, LN06a] (both parties confirm the SAS) while binding arbitrary messages (e.g., KEM public keys/encapsulations or context). We prove they UC-realize $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ and $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$, respectively. We also present a sketch of a variant of π_{SAS}^X without random oracles in Appendix F.
3. **Composable secure message transfer.** We show how composing SAS-based authentication with standard encryption primitives (KEM/DEM) yields a UC realization of secure message transfer $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$, preserving the explicit misbinding parameter under composition. We detail two practical instantiations: an SMT variant over signatures and an SMT variant over MACs in the Appendix A.
4. **Systematization of practice.** We systematize real-world tooling along the axes “how peers meet,” “what the human binds,” and “what the service learns” under a malicious-server assumption. For one-shot file transfer we contrast unauthenticated WebRTC data channels with PAKE-based “one-code” designs and our SAS-based alternative. We also survey deployed SAS usage and separate *session-bound SAS* (e.g., ZRTP, Matrix SAS, BLE Numeric Comparison) from *identity-key fingerprints* (e.g., Signal, WhatsApp, iMessage, Wire, Threema, Telegram).

The code and the reference application of the protocols π_{SAS}^X are available on Github⁹.

2 Related Work

A well-known and widely adopted paradigm for establishing a session key, that can be used for secure file transfer, is an Authenticated Key-Agreement (AKA). These protocols, however, tend to be complex, both in terms of their design and their target guarantees. They are typically built for environments with multiple parties that may act interchangeably as sender or receiver, and with corruption that can be static or adaptive. This complexity is not itself a showstopper. The core challenge, however, lies in the authentication component of most AKA protocols, which usually relies on a public-key infrastructure (PKI), an assumption that is not viable in our setting.

It is worth noting that there is a line of AKA protocols where authentication is achieved via Short Authentication Strings (SAS), which can be seen—roughly speaking—as fingerprints of the communication transcript. As shown by Vaudenay [Vau05], such mechanisms can securely realize the authentication step of an AKA under a weak, human-verifiable out-of-band (OOB) channel. We will return to SAS-based approaches below.

Another nearby alternative is Password-Authenticated Key Exchange (PAKE), which requires a password distribution phase. This phase is often avoided in discussions, but it implicitly assumes the existence of some secure channel, oftentimes an OOB channel with confidentiality and authenticity, for the safe transmission or registration of the password. This is precisely the kind of assumption we wish to avoid. Instead, we assume only an authenticated OOB channel for "short strings", which removes the need for confidentiality guarantees that PAKE's registration phase relies on.

A closely related device-onboarding scenario appears with low-power/IoT devices, where unassociated devices share no state and PKI setup may be costly or infeasible. To address this, works in the *Message Recognition Protocol* (MRP) family introduce a weaker primitive that authenticates message origin without providing full identity authentication or confidentiality. MRPs can bootstrap secure links in such constrained Settings [MV10, GMS10, HWGW05, BTT15]. These are adjacent to our goal but target a different abstraction and guarantee set.

SAS and out-of-band (OOB) channels. Authenticating a session by comparing a short authenticated string (SAS) over an auxiliary, human-verifiable channel originates with Vaudenay [Vau05], who formalized SAS-based authentication and gave commitment-style protocols that remain secure against attackers who may delay, drop, or replay the OOB message but can only cause a misbinding with

⁹ <https://github.com/collapsinghierarchy/noisytransfer-protocol> and <https://github.com/collapsinghierarchy/noisytransfercli>

probability at most 2^{-t} for t -bit SAS. Subsequent work optimized and generalized SAS for pairing and key agreement: Laur–Nyberg’s MANA/MA-DH and MANA-IV families [LN06a], Pasini–Vaudenay’s non-interactive message authentication [PV06a], and group extensions of SAS-based AKE [LP08]. In particular, Laur and Pasini’s consolidated treatment [LP09] describes the MANA IV pattern, i.e., commitment, challenge, and opening, followed by a bidirectional SAS comparison, for mutual data authentication. Our protocols π_{SAS} and π_{SAS}^X follow the MANA IV mechanics, thus we generalize the bound data from Diffie–Hellman shares to arbitrary messages and cast the outcome as realizations of explicit authenticated-channel functionalities (one-sided and mutual) that we later compose into secure message transfer.

Formal security for SAS protocols. Most of the SAS literature proves security in bespoke, game-based models tailored to pairing/AKE (typically commitment-based, sometimes in the random-oracle model), including SAS-based message authentication and AKE with key-reuse constraints [LN06b, PV06a, JS10]. These works bound an active attacker’s success by 2^{-t} for a t -bit display and provide concrete, protocol-specific guarantees. However, to the best of our knowledge there is no treatment that (i) defines a UC authenticated-channel ideal functionality with an explicit misbinding parameter ε , (ii) shows that an SAS-based protocol realizes such a functionality in a modern UC framework, and (iii) composes this into higher-level secure message transfer while preserving the explicit ε under composition. A related line (e.g., [LP09]) studies user-aided security outside the modern UC framework and does not expose the OOB/SAS channel as a first-class ideal functionality with parameter ε . Closer to modern UC, Benin–Toledo–Tromer [BTT15] model message-recognition protocols in UC using an ideal comparison primitive and focus on authentication bootstrapping (one human check, then continue) without confidentiality (termed “privacy” there). They also do not parameterize the residual MitM guess probability as an explicit ε carried through composition. This is the gap our $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$, $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R} / \mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}$, and $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ treatment fills.

Beyond pure SAS: richer OOB functionalities. Industry onboarding protocols often rely on richer OOB channels (e.g., QR/NFC) that carry long bootstrapping material (public keys, credentials) rather than a short digest. Examples include Wi-Fi Easy Connect / Device Provisioning Protocol (DPP) where onboarding proceeds by scanning a QR code or tapping NFC, and the IETF’s BRISKI for zero-touch bootstrapping [And24, PR⁺21]. These approaches assume stronger OOB capabilities and trust roots (factory-installed credentials, manufacturer CAs) than our low-bandwidth SAS setting. They are therefore complementary, not substitutes: our work explicitly targets the minimal, synchronous, human-verifiable OOB channel with immediate delivery for short strings.

3 Preliminaries

3.1 Notation

We model $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ as a random oracle; for $t \leq n$ let $\text{Trunc}_t : \{0, 1\}^n \rightarrow \{0, 1\}^t$ be fixed-bit truncation. We write $\text{sas}_t(x) = \text{Trunc}_t(H(x))$. $\text{Ber}(\varepsilon)$ denotes a Bernoulli random variable with success probability ε . For a finite set S , we write $x \xleftarrow{\$} S$ to denote that x is sampled uniformly at random from S .

3.2 Cryptographic Primitives

- $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ is an **IND-CPA-secure** key-encapsulation mechanism.
- $\text{DEM} = (\text{Enc}, \text{Dec})$ is an **OT-CPA-secure** data-encapsulation (symmetric) cipher.
- $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Ver})$ is an **EUFCMA-secure** digital-signature scheme.

One-time IND-CPA security for a DEM. Let $\text{DEM} = (\text{ENC}, \text{DEC})$ be a symmetric encryption scheme with keyspace $\mathcal{K}$. We define the one-time indistinguishability experiment against an adversary $\mathcal{A}$:

Definition 1 (OT-CPA for randomized DEM [KL20]). *In experiment $\text{Exp}_{\text{DEM}}^{\text{OT-CPA}}(\mathcal{A})$:*

1. *The challenger samples $K \xleftarrow{\$} \mathcal{K}$ and $b \xleftarrow{\$} \{0, 1\}$.*
2. *$\mathcal{A}$ outputs two equal-length messages (m_0, m_1)*
3. *The challenger samples fresh internal randomness r for ENC and returns $c \leftarrow \text{ENC}_K(m_b; r)$ to $\mathcal{A}$.*
4. *$\mathcal{A}$ outputs a bit b' .*

The advantage is

$$\text{Adv}_{\text{DEM}}^{\text{OT-CPA}}(\kappa) := \max_{\mathcal{A}} \left| \Pr[\text{Exp}_{\text{DEM}}^{\text{OT-CPA}}(\mathcal{A})=1] - \frac{1}{2} \right|,$$

where $\text{Exp}_{\text{DEM}}^{\text{OT-CPA}}(\mathcal{A})=1$ iff $b'=b$. Security requires $\text{Adv}_{\text{DEM}}^{\text{OT-CPA}}(\kappa) = \text{negl}(\kappa)$.

Commitments

Definition 2 (Commitment scheme). *A commitment scheme is a tuple of PPT algorithms $(\text{Commit}, \text{Verify})$ with message space $\mathcal{M}$ and randomness space $\mathcal{R}$, where*

$$(C, d) \leftarrow \text{Commit}(m; r)$$

on input $m \in \mathcal{M}$ and randomness $r \xleftarrow{\$} \mathcal{R}$ produces a commitment C and de-commitment witness d . The deterministic $\text{Verify}(C, m, d)$ outputs a bit in $\{0, 1\}$, accepting valid openings.

Correctness. For all $m \in \mathcal{M}$,

$$\Pr_{r \leftarrow \$\mathcal{R}} [\text{Verify}(C, m, d) = 1 \mid (C, d) \leftarrow \text{Commit}(m; r)] \geq 1 - \text{negl}(\lambda),$$

i.e., honestly generated commitments open successfully except with negligible probability.

Computational binding. For every PPT adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} (C, m, m', d, d') \leftarrow \mathcal{A}(1^\lambda) : m \neq m' \\ \text{Verify}(C, m, d) = 1 \wedge \text{Verify}(C, m', d') = 1 \end{array} \right] \leq \text{negl}(\lambda).$$

That is, no efficient adversary can produce a single commitment along with two different valid openings.

Computational hiding. For every PPT adversary $\mathcal{A}$, the advantage in the following experiment is negligible:

1. $\mathcal{A}$ outputs two messages $m_0, m_1 \in \mathcal{M}$ of equal length.
2. A random bit $b \leftarrow \$\{0, 1\}$ and randomness $r \leftarrow \$\mathcal{R}$ are chosen, and $C \leftarrow \text{Commit}(m_b; r)$ is computed.
3. $\mathcal{A}$ is given C and outputs a guess b' .

The hiding advantage is

$$|\Pr[b' = b] - \frac{1}{2}| \leq \text{negl}(\lambda).$$

Thus, commitments to m_0 and m_1 are computationally indistinguishable.

In our main constructions we instantiate `Commit` as hash-then-open in the random-oracle model (binding and computationally hiding under [BR93]).

3.3 Universal Composability Framework

The UC framework [Can20, Can01] additionally introduces an interactive Turing machine $\mathcal{Z}$, called the environment, which has to distinguish the real world execution from the ideal world simulation by actively communicating with the adversary during the function computation. Additionally, we define the environment's view with an adversary and a function as the input, the exchanged messages between the corrupted parties and adversary together with the outputs. This restricts the simulation-based proof since rewinding of the adversary is not possible in this case. For a formal statement, we recite the simulation paradigm of the UC framework by Canetti.

Definition 3 ([Can20]). Let π be a protocol on inputs $\mathbf{x} = (x_1, \dots, x_n) \in X^n$ for n parties and $\mathcal{F}$ a functionality. We say that π UC-securely realizes $\mathcal{F}$ if for all polynomially bounded adversaries $\mathcal{A}$ a simulator $\mathcal{S}$ exists such that for every polynomially bounded environment $\mathcal{Z}$:

$$\text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}(\lambda, \mathbf{x})_{\lambda \in \mathbb{N}, \mathbf{x} \in X^n} \stackrel{c}{\approx} \text{REAL}_{\pi, \mathcal{A}, \mathcal{Z}}(\lambda, \mathbf{x})_{\lambda \in \mathbb{N}, \mathbf{x} \in X^n},$$

where $REAL_{\pi, \mathcal{A}, \mathcal{Z}}(\lambda, \mathbf{x})_{\lambda \in \mathbb{N}, \mathbf{x} \in X^n}$ is the environment's view when interacting with $P_1, \dots, P_n$ and $\mathcal{A}$ and $IDEAL_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}(\lambda, \mathbf{x})_{\lambda \in \mathbb{N}, \mathbf{x} \in X^n}$ is $\mathcal{Z}$'s view when interacting with $\mathcal{F}$ and $\mathcal{S}$.

Additionally, the Universal Composition framework has the property that any UC-secure protocol can be securely (concurrently) composed with any other UC-secure protocol. This is the universal composition theorem, which is rephrased here without proof.

Theorem 1 (UC composition theorem, see [Can20]). *Let $\mathcal{F}$ and $\mathcal{G}$ be two ideal functionalities. Further let ρ be a protocol that securely UC-realizes $\mathcal{G}$ and let π be a protocol that securely UC-realizes $\mathcal{F}$. Then the composed protocol ρ^π securely UC-realizes $\mathcal{G}$.*

We refer the proof to [Can20].

3.4 Ideal Functionalities

The following authenticated channel is included only as a warm-up: it motivates the first step in proving a simplified variant of our SAS-based authentication protocol. It is the standard $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}$ from Canetti [Can20], extended with a bounded active-forgery hook to capture the residual success probability from random guessing a short authentication string. Concretely, in $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ we let the adversary (i) schedule delivery and (ii) make a single online forgery attempt per session that succeeds with probability ε (e.g., $\varepsilon = 2^{-t}$ for a t -bit SAS). This abstraction serves as a stepping stone toward our cross-authentication functionality and the full SAS-based secure channel. For the sake of simplicity we ignore any corruptions in a 2-party setting.

Ideal Functionality $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$

Parameters. Forgery bound $\varepsilon \in [0, 1]$.

Parties. A sender S , a receiver R , and the adversary $\mathcal{A}$. We assume S and R remain honest.

State per sid: $m \leftarrow \perp$; $R \leftarrow \perp$; $\text{forged}_{\text{sid}} \in \{\perp, \text{false}, \text{true}\}$ (init $\perp$).

- **Delayed Send.** Upon input (Send, sid, R , m) from S , record (m , R), set $\text{forged}_{\text{sid}} \leftarrow \perp$, and send back-door (Sent, sid, S , R , m) to $\mathcal{A}$.
- **Active Forgery (one online attempt).** Upon back-door (Forge, sid, m' , R) from $\mathcal{A}$ (at most once per sid): sample $b \leftarrow \text{Ber}(\varepsilon)$; send (ForgeResult, sid, b) to $\mathcal{A}$; set $\text{forged}_{\text{sid}} \leftarrow (\text{true if } b=1 \text{ else false})$; if $b=1$, also record (m' , R) for this sid.
- **Release.** Upon back-door (ok, sid) from $\mathcal{A}$:
 - If $\text{forged}_{\text{sid}} = \text{true}$, output (Sent, sid, S , m') to R .
 - Else if $\text{forged}_{\text{sid}} = \perp$ (no attempt), output (Sent, sid, S , m) to R .
 - Else ($\text{forged}_{\text{sid}} = \text{false}$; attempted and failed), **output nothing**.
- **Ignore** all other inputs and back-door messages.

Cross-Authentication with a Bernoulli Forgery-Gate We continue to assume that S and R are honest. In the next step we lift $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ to a two-sided, user-aided cross-authentication functionality $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ in the sense of Laur–Pasini [LP09]: each party contributes a short context string (e.g., ephemeral identifiers or public parameters), the adversary receives the prescribed leakage and may schedule delivery, and is granted a single online misbinding attempt that succeeds with probability ε (capturing random SAS guessing). On acceptance, both parties obtain the peer’s authenticated contribution, which we later use as input to the SAS-based secure channel.

Ideal Functionality $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$

Parameters. Forgery bound $\varepsilon \in [0, 1]$.

Parties. Honest S and R , and the adversary $\mathcal{A}$.

State per sid. $m_S \leftarrow \perp$, $m_R \leftarrow \perp$; **forged** $\in \{\perp, \text{true}, \text{false}\}$ (init $\perp$); $m_S^{\text{fg}}, m_R^{\text{fg}} \leftarrow \perp$; **delivered** $\leftarrow \text{false}$.

- **Propose (from S).** On (Propose, sid, R , m_S): record m_S ; send back-door (Seen, sid, S , m_S) to $\mathcal{A}$.
- **Respond (from R).** On (Respond, sid, S , m_R): record m_R ; send back-door (Seen, sid, R , m_R) to $\mathcal{A}$.
- **Active Forgery (one online attempt).** At any time *before* delivery, upon back-door (Forge, sid, m'_S, m'_R) (at most once per sid): sample $b \leftarrow \text{Ber}(\varepsilon)$; send (ForgeResult, sid, b) to $\mathcal{A}$. If $b = 1$, set $(m_S^{\text{fg}}, m_R^{\text{fg}}) \leftarrow (m'_S, m'_R)$ and **forged** $\leftarrow \text{true}$; else set **forged** $\leftarrow \text{false}$.
- **Release.** Upon back-door (ok, sid) from $\mathcal{A}$, if **delivered** = false and $m_S \neq \perp$ and $m_R \neq \perp$ then:
 - If **forged** = true, output (XAuth, sid, S , m_S^{fg}) to R and (XAuth, sid, R , m_R^{fg}) to S ; set **delivered** $\leftarrow \text{true}$.
 - Else if **forged** = $\perp$ (no attempt), output (XAuth, sid, S , m_S) to R and (XAuth, sid, R , m_R) to S ; set **delivered** $\leftarrow \text{true}$.
 - Else (**forged** = false; attempted and failed), **output nothing** and set **delivered** $\leftarrow \text{true}$.
- **Ignore** all other inputs/back-door messages.

Secure Message Transfer with a Bernoulli MitM-Gate The next functionality refines secure message transmission to account for the non-negligible online success of an active adversary under SAS/PAKE-style human authentication, in the same spirit as our authenticated-channel variants. We constrain leakage to message length only, i.e., $\ell(m) = |m|$. Classical UC $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ delivers the sender’s message intact once the adversary schedules delivery, which over-idealizes SAS/PAKE-backed channels: an attacker may achieve a *MitM misbinding* with probability essentially $\varepsilon \approx 2^{-t}$ for a t -bit SAS (up to standard concurrency factors), or roughly $1/|\mathcal{D}|$ per online guess in the PAKE setting [Vau05, PV06b, Exc05]. Following Canetti–Halevi–Katz’s UC treatment of low-entropy secrets [Exc05], we adopt the “explicit guessing power” modeling and

build a single Bernoulli(ε) gate into the functionality. Concretely, the adversary gets exactly one online misbinding attempt per session, reflecting a single SAS comparison; if it succeeds, confidentiality and authenticity fail for that session (the original m is learned) and the adversary may substitute the delivered message, with delivery still gated by (ok, sid).

Functionality $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$
Parameters. Misbinding success bound $\varepsilon \in [0, 1]$; leakage $\ell(m) = m$. Parties. Honest S and R, and the adversary $\mathcal{A}$. State per sid: $m \leftarrow \perp$; $R \leftarrow \perp$; $\text{auth} \in \{\perp, \text{true}, \text{false}\}$ (init $\perp$); $m^{\text{mitm}} \leftarrow \perp$; $\text{delivered} \leftarrow \text{false}$. • Send. On (Send, sid, R, m) from S: record m and R; set $\text{auth} \leftarrow \perp$ and $m^{\text{mitm}} \leftarrow \perp$; send back-door (Sent, sid, $S, R, \ell(m)$) to $\mathcal{A}$. • Active MitM. Before delivery and at most once per sid, upon back-door (MisbindTry, sid): sample $b \leftarrow \text{Ber}(\varepsilon)$; send (MisbindResult, sid, b) to $\mathcal{A}$. – If $b = 1$, set $\text{auth} \leftarrow \text{true}$ and send back-door (Reveal, sid, m) to $\mathcal{A}$. – Else set $\text{auth} \leftarrow \text{false}$. • Set substituted payload. At any time before delivery, upon back-door (SetMitmPayload, sid, m'): if $\text{auth} \neq \text{true}$ then ignore; else require $m' = m$ (else ignore); if $m^{\text{mitm}} = \perp$ then set $m^{\text{mitm}} \leftarrow m'$. • Delivery (gated by ok). Upon back-door (ok, sid) from $\mathcal{A}$, if $\text{delivered} = \text{false}$ then: – If $\text{auth} = \text{true}$ and $m^{\text{mitm}} \neq \perp$, output (Sent, sid, S, m^{mitm}) to R and set $\text{delivered} \leftarrow \text{true}$. – Else if $\text{auth} = \text{false}$ (attempted and failed), output nothing and set $\text{delivered} \leftarrow \text{true}$. – Else ($\text{auth} = \perp$; no attempt), output (Sent, sid, S, m) to R and set $\text{delivered} \leftarrow \text{true}$. • Ignore all other inputs and back-door messages.

On Strengthening $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ Our current ideal functionality $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ leaks the sender's message m to the adversary on a successful MisbindTry, modeling the *worst case* (that we can't protect from with probability ε) where the receiver's public key is subverted ($\text{pk}_R \rightsquigarrow \text{pk}'_R$) so that the real attacker can decapsulate and learn m . This choice is simulatable and faithful for that attack surface, but it is conservative: there are executions with *sender-only* misbinding (where vk_S is swapped yet pk_R remains honest) in which the real attacker *cannot* learn m .

A stricter idealization would remove the Reveal leakage entirely: the Bernoulli gate still models one online MitM attempt, but even on success the functionality never discloses m to the adversary. Such a variant appears simulatable only under an extra guarantee that the receiver's KEM public key is *not* subverted (while sender verification-key substitution may still occur). In our SAS/no-setup

setting this seems unattainable without additional trust (e.g., PKI or channel binding).

4 Universally Composable Authentication from SAS

This section puts SAS-based authentication on a UC footing. We expose a hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ channel for short, human-verifiable comparison and give two compact protocols, π_{SAS} and π_{SAS}^X , that follow a commit-respond-open pattern with a single SAS check. We prove they UC-realize $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ and $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ with $\varepsilon = 2^{-t}$ in the ROM.

4.1 Out-of-Band SAS Confirmation

One crucial component of our protocol is the authenticated *out-of-band channel* (OOB), which we model as a hybrid functionality and follow the modelling from [BTT15]. Our abstraction captures the synchronous user interaction between the Sender and Receiver required to verify the SAS.

Ideal Functionality $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$

Parties: R (receiver) and S (sender) and the adversary $\mathcal{A}$.

- **SAS Comparison:** Upon input $(\text{SAS}, \text{sid}, s)$ from S or R , store the SAS and send $(\text{SAS}, \text{sid}, s)$ to $\mathcal{A}$. If the SAS from the other party was already received, i.e. s' , compute the flag $b := (s == s')$ then return to the sender and receiver $(\text{SAS}, \text{sid}, b)$ (immediate delivery).
- **Ignore** all other inputs and back-door messages.

This channel differs from the usual model of data transmission over untrusted networks, where an adversary can arbitrarily drop messages. In our $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ hybrid, we assume the SAS is displayed to honest users on uncompromised hardware and software, so message delivery cannot be suppressed. Addressing a compromised display or denial-of-service attack on the OOB channel is an interesting direction for future work, but we leave it outside the scope of this paper.

Asynchronous OOB confirmation. Some deployments cannot assume co-presence. An asynchronous variant of $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ models delivery of the SAS values and a subsequent approve/deny bit over an authenticated but delay-tolerant channel (for example, an end-to-end encrypted messenger or some asynchronous call). To preserve security, the SAS must be bound to a unique session identifier, peer descriptors, and freshness data, and the functionality must reject reuse or replay of approvals. Compared to synchronous display-and-compare, the enhanced functionality shifts assurance to the authenticity of the external channel and increases the risk of misbinding unless the UI surfaces clear context (who, which device, when). The construction remains compatible with our SAS-based authentication, but in such settings a PAKE workflow can be operationally attractive because the user performs an initial single out-of-band password exchange while the client automates the rest. A formal treatment of an asynchronous variant of $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ is, however, out of scope here and will be subject to future work.

Human error. A central practical risk in human-assisted binding is misbinding/unknown key-share (UKS): a compromised endpoint or a confused user may bind to the wrong peer despite non-matching SAS. Empirical studies show that the representation and length of the value users compare materially affect error rates, task time, and success (e.g., decimal digits vs. alternatives) [KST⁺09, KSTU09]. Surveys of secure device pairing report similar trends across modalities and settings [FÁS⁺18]. Different OOB carriers are possible, i.e., visual/QR, audio, NFC, motion, etc., with differing reliability and cognitive load [MPR05, FÁS⁺18]. In our modeling, $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ can be enriched with a parameter $\varepsilon_{\text{human}}$ capturing human fallibility (misreads, inattentive comparison, UI confusion), meaning that even on unequal transcripts a user may still accept with probability $\varepsilon_{\text{human}}$. By contrast, the ε that appears in $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}/\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}$ quantifies the cryptographic random-guess surface for an active MitM under correct user behavior (e.g., 2^{-t} for a t -bit SAS). We keep these knobs separate. In the proofs we set $\varepsilon_{\text{human}}=0$ to isolate the cryptographic guarantees and leave a calibrated treatment of human error to future work.

4.2 Universally Composable SAS and OOB based One-Sided Authentication

We begin with a minimal, user-aided authentication primitive: a three-move SAS protocol that binds a sender’s short context string to a receiver, using a single human comparison. The hash H is modeled as a random oracle and the displayed short authentication string (SAS) is the t -bit truncation of H on the session transcript. We will argue that the adversary’s best active success is $\varepsilon = 2^{-t}$, up to standard concurrency factors [Vau05, PV06b, LP09]. The human check is abstracted by the ideal functionality $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$, and we keep the network parties honest.

Real Protocol π_{SAS} (hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$)

Parties. S , R , the adversary $\mathcal{A}$, and hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$.

Parameters. Output length $t \in \mathbb{N}$. Let H be a random oracle and $\text{SAS}(x) := \text{Trunc}_t(H(x))$.

0. Session setup (SID).

Either receive a session identifier sid from the calling environment, or (if not provided) have S sample a fresh $\text{sid} \leftarrow \$ \{0, 1\}^\kappa$ and notify R over the main channel. All local state and every call to subroutines/functionalities in this session is tagged with the same sid .

1. Commitment. $S \rightarrow R$: $(C, d) \leftarrow \text{Commit}(\text{sid} \parallel m_S; r_S)$, where r_S are the sender’s random coins (contained in d); send C .

2. Receiver’s Contribution. $R \rightarrow S$: sample $r_R \leftarrow \$ \{0, 1\}^\kappa$.

3. Reveal & SAS. $S \rightarrow R$: send $\text{Open}(C, d)$.

If R rejects the opening, abort. Otherwise both compute

$$s := \text{SAS}(\text{sid} \parallel m_S \parallel r_S \parallel r_R)$$

and each party sends $(\text{SAS}, \text{sid}, s)$ to $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$.

4. Confirmation. $\mathcal{F}_{\text{OOB}}^{\text{SAS}} \rightarrow \{S, R\} : (\text{SAS}, \text{sid}, b)$.
 R outputs m_S iff $b = 1$; otherwise abort.

Theorem 2 (UC security of π_{SAS} toward $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$). *Let $t \in \mathbb{N}$ and define $\varepsilon := 2^{-t}$. Assume: (i) $\text{Com} = (\text{Commit}, \text{Open})$ is computationally hiding and binding; (ii) H is modeled as a random oracle and $\text{SAS}(x) = \text{Trunc}_t(H(x))$; (iii) the hybrid functionality $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ (SAS comparison with leakage to the adversary) is available. Then for every PPT adversary $\mathcal{A}$ there exists a PPT simulator $\mathcal{S}$ such that for every PPT environment $\mathcal{Z}$,*

$$\text{EXEC}_{\mathcal{Z}, \mathcal{A}, \Pi_{\text{SAS}}}^{\mathcal{F}_{\text{OOB}}^{\text{SAS}}} \approx \text{IDEAL}_{\mathcal{Z}, \mathcal{S}, \mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)}.$$

Hence, in the $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ -hybrid, Π_{SAS} UC-realizes the authenticated one-way channel with a single online active-forgery attempt that succeeds with probability ε .

At a high level, in the ideal world the simulator receives the sender's input m_S from $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ via the back-door tape. Using this, it precomputes the honest transcript (including the t -bit SAS) and emulates the network and $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ leakage exactly, preserving the real execution's scheduling. The only remaining distinguishing avenue is an active forgery/misbinding. Via a short hybrid sequence (commitment hiding/binding and RO unpredictability and programming) we bound its success by $\varepsilon = 2^{-t}$ and couple it to the Bernoulli(ε) gate of $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$.

hybrid argument. We work with the standard dummy-adversary reduction and indistinguishability of hybrids in the UC sense (cf. Canetti [Can20]). We keep S, R honest throughout; the adversary controls network scheduling and sees the $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ leakage.

Notation. Fix a session identifier sid chosen by the environment. The sender's payload is m_S ; the receiver samples $r_R \leftarrow \$ \{0, 1\}^\kappa$; the sender's commitment coins are $r_S \leftarrow \$ \{0, 1\}^\kappa$. We model H as a random oracle with output length $n = n(\kappa)$ and write $\text{Trunc}_t : \{0, 1\}^n \rightarrow \{0, 1\}^t$. Unless stated otherwise,

$$s := \text{Trunc}_t(H(\text{sid} \parallel m_S \parallel r_S \parallel r_R)).$$

Random-oracle table. We implement H via *lazy sampling* using a global partial map

$$T : \{0, 1\}^* \rightharpoonup \{0, 1\}^n,$$

initially empty and shared across all ITMs in the execution (parties, adversary, and simulator). On a query x to H : if $x \in \text{dom}(T)$ return $T[x]$; otherwise sample

$y \leftarrow \$ \{0, 1\}^n$ uniformly, set $T[x] \leftarrow y$, and return y . We say “*program T at x* ” to mean assigning $T[x]$ to some value (possibly under a t -bit prefix constraint in the coupling step), always without overwriting existing entries. Throughout, $x \in T$ abbreviates $x \in \text{dom}(T)$.

For the SAS comparison we will form

$$x_R := (\text{sid} \parallel m'_S \parallel r'_S \parallel r_R) \quad \text{and} \quad x_S := (\text{sid} \parallel m_S \parallel r_S \parallel \tilde{r}_R),$$

taken from the values actually seen on R 's and S 's wires, respectively. We use the event $\text{preHit}_{\text{sid}} := [x_R \in T \vee x_S \in T]$ to denote a *pre-reveal RO hit* in session sid . The commitment's decommitment is denoted d .

Game H0 (Real). The real execution of Π_{SAS} in the $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ -hybrid against $\mathcal{A}$.

H1 (Code lifting). We introduce an intermediate wrapper that internally runs *all* honest parties and the hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$, and forwards wires to $\mathcal{A}$ and $\mathcal{Z}$ verbatim. This is the usual “all-powerful simulator shell” that only reorganizes code, not distributions.

Lemma 1. $\text{EXEC}[\text{H0}] \equiv \text{EXEC}[\text{H1}]$ (*perfectly indistinguishable*).

H2 (Hiding-based replacement of the commitment). In H2, the wrapper replaces the real $C \leftarrow \text{Commit}(\text{sid} \parallel m_S; r_S)$ by a uniform string $C \leftarrow \$ \{0, 1\}^{n(\kappa)}$, while still opening it later to $(\text{sid} \parallel m_S, d)$ as in the real protocol (i.e., it stores a table to ensure a consistent decommitment). Collisions among sampled C are ruled out with overwhelming probability via the birthday bound.

Lemma 2 (Commitment hiding). *If Com is computationally hiding, then $\text{EXEC}[\text{H1}] \approx \text{EXEC}[\text{H2}]$.*

Sketch. A standard reduction: a distinguisher between H1 and H2 yields a PPT adversary that breaks hiding by telling apart a commitment to m_S from a random string placed on the same wire. $\square$

H3 (Abort on BAD double-opening). From H2, define H3 that aborts and outputs a special symbol **Bad** if the adversary ever induces a valid *second* opening $(m', d') \neq (m_S, d)$ for the same C .

Lemma 3 (Commitment binding). *If Com is binding then $\Pr[\text{Bad}]$ is negligible, hence $\text{EXEC}[\text{H2}] \approx \text{EXEC}[\text{H3}]$.*

H4^{pre} (Flag pre-reveal RO hits). H4^{pre} is identical to H3 except it sets a boolean flag $\text{preHit}_{\text{sid}} \leftarrow \text{true}$ if, *before* the SAS comparison in session sid , the adversary has queried the RO at either x_R or x_S (as defined later in H4), i.e., if $x_R \in T$ or $x_S \in T$ holds at SAS time. Execution proceeds unchanged.

Lemma 4 (Pre-reveal RO hits are negligible). *Let $r_S \leftarrow \$\{0,1\}^\kappa$ be sampled and hidden inside the sender's commitment (so $x_R := (\text{sid} \parallel m'_S \parallel r'_S \parallel r_R)$ and $x_S := (\text{sid} \parallel m_S \parallel r_S \parallel \tilde{r}_R)$ contain r_S). Then for any PPT adversary making $q_H(\kappa)$ RO queries,*

$$\Pr[\text{preHit}_{\text{sid}}] \leq 2q_H(\kappa) \cdot 2^{-\kappa} = \text{negl}(\kappa).$$

Consequently, $\text{EXEC}[\text{H3}] \approx \text{EXEC}[\text{H4}^{\text{pre}}]$.

Proof. Before reveal, (m_S, r_S) is hidden; in particular r_S is uniform and unknown. Hitting x_R (or x_S) with an RO query requires guessing r_S , which succeeds with probability at most $q_H \cdot 2^{-\kappa}$ per input. A union bound over $\{x_R, x_S\}$ gives the claim. $\square$

H4 (Flag tamper and identical-until-bad). H4 is identical to H3 except it sets a boolean flag $\text{tampered}_{\text{sid}} \leftarrow \text{true}$ if, in session sid , the adversary alters either (i) the sender's first message (a valid commitment C' is placed on $S \rightarrow R$ that differs from the honest C), or (ii) the receiver's nonce in flight ($\tilde{r}_R \neq r_R$ is placed on $R \rightarrow S$). At the SAS comparison, define

$$s_R := \text{Trunc}_t(H(\text{sid} \parallel m'_S \parallel r'_S \parallel r_R)), \quad s_S := \text{Trunc}_t(H(\text{sid} \parallel m_S \parallel r_S \parallel \tilde{r}_R)),$$

where in the non-tampered case $\tilde{r}_R = r_R$, $m'_S = m_S$ and $r'_S = r_S$. Let the *bad* event be $\text{bad}_{\text{sid}} := [\text{tampered}_{\text{sid}} \wedge (s_R = s_S)]$. Execution proceeds unchanged (no abort); we only *record* bad_{sid} .

Lemma 5 (SAS equality under tampering). *Model H as a random oracle. Conditioned on $\text{tampered}_{\text{sid}}$ and on the commitment being binding to m_S , the two inputs $(\text{sid} \parallel m'_S \parallel r'_S \parallel r_R)$ and $(\text{sid} \parallel m_S \parallel r_S \parallel \tilde{r}_R)$ are distinct with overwhelming probability; hence s_R and s_S are independent uniform t -bit strings. Therefore, $\Pr[\text{bad}_{\text{sid}}] \leq 2^{-t} + \text{negl}(\kappa)$, and by the fundamental lemma of game-playing,*

$$\left| \Pr[\text{EXEC}[\text{H3}] \in \mathcal{V}] - \Pr[\text{EXEC}[\text{H4}] \in \mathcal{V}] \right| \leq \Pr[\text{bad}_{\text{sid}}] \leq 2^{-t} + \text{negl}(\kappa)$$

for any PPT test $\mathcal{V}$.

Proof. Binding fixes m_S (and r_S if included) from R 's perspective once C is opened honestly. If the adversary injects a different commitment C' , it must also provide a valid opening to make R proceed; except with negligible probability this opening yields $(m'_S, r'_S) \neq (m_S, r_S)$, thereby diverging from S 's view. The only remaining effective tamper point is delivering $\tilde{r}_R \neq r_R$ to S , which then diverges from R 's view that uses r_R .

In the random-oracle model, distinct inputs yield independent uniform outputs; after truncation, $\Pr[s_R = s_S] = 2^{-t}$. Any residual probability mass is due only to violating binding or forcing an RO collision on distinct inputs, both of which are negligible in κ . $\square$

H5 (Coupling to the ideal Bernoulli gate and Prefix-constrained lazy RO). We define a simulator $\mathcal{S}$ that, on top of H4's code lifting, replaces the real network and $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ by a perfect emulation driven by the ideal functionality $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$, while maintaining a lazy-sampled RO table T .

- **On (Sent, sid, S , m_S) from $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$:** $\mathcal{S}$ starts a local simulation of the real transcript: sample $C \leftarrow \$ \{0, 1\}^{n(\kappa)}$, $r_R \leftarrow \$ \{0, 1\}^\kappa$ (and any sender coins embedded in the commitment), relay C on $S \rightarrow R$, then (under adversarial scheduling) relay r_R on $R \rightarrow S$, then the opening (m_S, r_S) on $S \rightarrow R$. No accept bit is returned yet; $\mathcal{S}$ only mirrors $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$'s leakage interface later at SAS time.
- **At the SAS comparison (semantic tamper test):** Form

$$x_R := (m'_S \parallel r'_S \parallel r_R), \quad x_S := (m_S \parallel r_S \parallel \tilde{r}_R),$$

where x_R uses the actually received opening on R 's side and x_S uses whatever $\tilde{r}_R$ arrived at S .

- *If $x_R = x_S$ (no tamper):* program T lazily at x_R (if needed), let both parties' SAS be $s := \text{Trunc}_t(H(x_R))$, leak $(\text{SAS}, \text{sid}, s)$ to $\mathcal{A}$ for each party, and return $b=1$ from the emulated $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$. Later, upon (ok, sid) from $\mathcal{A}$, forward it to $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ so that it delivers $(\text{Sent}, \text{sid}, S, m_S)$ to R .
- *If $x_R \neq x_S$ (tamper):* invoke the ideal gate by sending $(\text{Forge}, \text{sid}, m', R)$ to $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ (where m' is the message determined by the adversarial flow) and receive $(\text{ForgeResult}, \text{sid}, b)$. Now program the RO lazily with a t -bit prefix constraint consistent with T :
 - * If neither x_R nor x_S is in T : sample $y_R, y_S \leftarrow \$ \{0, 1\}^n$ uniformly subject to $\text{Trunc}_t(y_R) = \text{Trunc}_t(y_S)$ iff $b=1$; set $T[x_R] \leftarrow y_R, T[x_S] \leftarrow y_S$.
 - * If exactly one is in T : sample the other uniformly subject to the same t -bit relation to the existing one and add it to T .
 - * If both are already in T : if the existing t -bit relation contradicts b , this is precisely the negligible pre-reveal hit isolated in Lemma 4; otherwise proceed.
 Let $s_R = \text{Trunc}_t(T[x_R])$, $s_S = \text{Trunc}_t(T[x_S])$; leak $(\text{SAS}, \text{sid}, s_R)$ and $(\text{SAS}, \text{sid}, s_S)$ to $\mathcal{A}$ as $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ would, and return that same bit b from the emulated $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ to both parties.
- **When $\mathcal{A}$ later sends (ok, sid) :** If no successful forgery occurred (i.e., $b \neq 1$ in the tampered case), forward (ok, sid) to $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ to deliver $(\text{Sent}, \text{sid}, S, m_S)$ to R .

Crucially, in the real world, any successful misbinding with honest parties requires that the two t -bit SAS values coincide on *distinct* RO inputs. By Lemma 5, this occurs with probability at most $\varepsilon + \text{negl}(\kappa)$, where $\varepsilon = 2^{-t}$. The coupling above replaces this event by the Bernoulli(ε) gate of $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ and mirrors the same accept/reject outcome in the emulated $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$, while preserving scheduling and leakage.

Lemma 6. $\text{EXEC}[\text{H5}] \approx \text{IDEAL}[\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon), \mathcal{S}]$.

Conclusion. By transitivity of indistinguishability across H0–H5, we obtain the claim of Theorem 2.

Final distinguishing bound. Let $\Delta(\kappa)$ denote the UC distinguishing advantage of any PPT $(\mathcal{A}, \mathcal{Z})$ pair between $\text{EXEC}_{\mathcal{Z}, \mathcal{A}, \Pi_{\text{SAS}}}^{\mathcal{F}_{\text{OOB}}^{\text{SAS}}}$ and $\text{IDEAL}_{\mathcal{Z}, \mathcal{S}, \mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)}$. Then

$$\begin{aligned} \Delta(\kappa) \leq & \underbrace{\mathcal{A}_{\text{Com}}^{\text{hide}}(\kappa)}_{\text{H1} \rightarrow \text{H2}} + \underbrace{\mathcal{A}_{\text{Com}}^{\text{bind}}(\kappa)}_{\text{H2} \rightarrow \text{H3}} + \underbrace{\Pr[\text{preHit}_{\text{sid}}]}_{\leq 2 q_H(\kappa) 2^{-\kappa}} \\ & + \underbrace{\left| \Pr_{\text{real}}[\text{accept} \mid x_R \neq x_S] - \varepsilon \right|}_{\leq \text{negl}(\kappa) \text{ by Lemma 5, } \varepsilon = 2^{-t}} \end{aligned}$$

and thus

$$\Delta(\kappa) \leq \mathcal{A}_{\text{Com}}^{\text{hide}}(\kappa) + \mathcal{A}_{\text{Com}}^{\text{bind}}(\kappa) + 2 q_H(\kappa) 2^{-\kappa} + \text{negl}(\kappa) = \text{negl}(\kappa).$$

The full simulator is given in Appendix B. $\square$

Attempt-conditioned gate. We sample the Bernoulli gate with parameter $\varepsilon = 2^{-t}$ iff the run reaches a genuine tamper point at SAS time, namely $x_R \neq x_S$. Let $I \in \{0, 1\}$ indicate this event. In the real world, conditioned on $I=1$ the two RO inputs are distinct, so the accept probability is $\varepsilon \pm \text{negl}(\kappa)$; hence

$$\Pr[\text{accept}]_{\text{real}} = \Pr[I=1] \cdot \varepsilon \pm \text{negl}(\kappa).$$

In the ideal world, the simulator triggers the gate exactly when $I=1$, and the functionality returns $\text{Ber}(\varepsilon)$, yielding

$$\Pr[\text{accept}]_{\text{ideal}} = \Pr[I=1] \cdot \varepsilon.$$

Thus an adversary who “plays weak” (e.g., tampers rarely or avoids creating $x_R \neq x_S$) merely decreases $\Pr[I=1]$ in *both* worlds; it cannot alter the per-attempt success, which is fixed at ε . Consequently the overall success probabilities match up to the negligible terms already isolated in the hybrids (pre-hit and RO-collision events).

Unfortunately, $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}$ is yet not sufficient for our requirements of a user-friendly protocol (albeit theoretically we can now effectively compose multiple instances and provide “virtual” bi-directional authentication), which is the same argument that [BTT15] used in order to define the functionality $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(k)$, which is a straight forward extension of $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}$ in order to allow bi-directional “back-and-forth” communication of k authenticate messages between S and R .

4.3 Universally Composable Cross-Authentication from SAS

In this section we extend π_{SAS} to support *bi-directional* delivery of one application message per party. The change is minimal: the receiver R now *piggybacks* its message $\mathbf{m}_R$ alongside its fresh nonce $\mathbf{r}_R$ in Step 2. The SAS then commits

to the full transcript $(\text{sid}, \mathbf{m}_S, \mathbf{m}_R, \mathbf{r}_S, \mathbf{r}_R)$, so any tampering that alters what either party sees requires a collision on the truncated random-oracle output. The session identifier sid is carried throughout to ensure subroutine-respecting composition. In the following we show that this change does not contradict the security.

Real Protocol π_{SAS}^X (hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$)

Parties. S , R , the adversary $\mathcal{A}$, and hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$.

Parameters. Output length $t \in \mathbb{N}$. Let H be a random oracle and $\text{SAS}(x) := \text{Trunc}_t(H(x))$. Let sid be the session identifier of this instance.

1. Commitment. $S \rightarrow R$: $(C, d) \leftarrow \text{Commit}(\text{sid} \parallel \mathbf{m}_S; \mathbf{r}_S)$, where $\mathbf{r}_S$ are the commitment coins (contained in d); send C .

2. Receiver's Contribution.

$R \rightarrow S$: choose $\mathbf{m}_R$ and sample $\mathbf{r}_R \leftarrow \$ \{0, 1\}^\kappa$; send $(\mathbf{m}_R, \mathbf{r}_R)$.

3. Reveal & SAS. $S \rightarrow R$: send $\text{Open}(C, d)$.

If R rejects the opening, abort. Else both compute

$$s := \text{SAS}(\text{sid} \parallel \mathbf{m}_S \parallel \mathbf{m}_R \parallel \mathbf{r}_S \parallel \mathbf{r}_R).$$

and send $(\text{SAS}, \text{sid}, s)$ to $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$.

4. Confirmation. $\mathcal{F}_{\text{OOB}}^{\text{SAS}} \rightarrow \{S, R\}$: $(\text{SAS}, \text{sid}, b)$.

R outputs $\mathbf{m}_S$ and S outputs $\mathbf{m}_R$ iff $b = 1$; otherwise both abort.

Theorem 3 (UC security of π_{SAS}^X toward $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$). *Let $t \in \mathbb{N}$ and set $\varepsilon := 2^{-t}$. Assume: (i) $\text{Com} = (\text{Commit}, \text{Open})$ is computationally hiding and binding; (ii) H is modeled as a random oracle and $\text{SAS}(x) = \text{Trunc}_t(H(x))$; (iii) the hybrid functionality $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ is available. Then for every PPT adversary $\mathcal{A}$ there exists a PPT simulator $\mathcal{S}$ such that for every PPT environment $\mathcal{Z}$,*

$$\text{EXEC}_{\mathcal{Z}, \mathcal{A}, \pi_{\text{SAS}}^X}^{\mathcal{F}_{\text{OOB}}^{\text{SAS}}} \approx \text{IDEAL}_{\mathcal{Z}, \mathcal{S}, \mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)}.$$

Hence, in the $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ -hybrid, π_{SAS}^X UC-realizes cross-authentication $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ with a single online active-forgery attempt of success probability ε .

Proof. For the sake of brevity we only provide the most important differences to the previous proof.

– **SAS inputs (pair-aware).** At compare time use

$$x_R := (\text{sid} \parallel \mathbf{m}'_S \parallel \mathbf{m}_R \parallel \mathbf{r}'_S \parallel \mathbf{r}_R), \quad x_S := (\text{sid} \parallel \mathbf{m}_S \parallel \tilde{\mathbf{m}}_R \parallel \mathbf{r}_S \parallel \tilde{\mathbf{r}}_R).$$

Tampering is the semantic event $x_R \neq x_S$ (covers flips of $\mathbf{m}_R$ and/or $\mathbf{r}_R$ in transit in addition to commitment tamper).

- **Pre-hit hybrid.** $\text{preHit}_{\text{sid}} := [x_R \in T \vee x_S \in T]$ remains negligible; proof unchanged (requires guessing hidden r_S).
- **Tamper lemma.** Conditioned on binding and $x_R \neq x_S$, the RO outputs are independent uniform strings; hence $\Pr[\text{Trunc}_t(H(x_R)) = \text{Trunc}_t(H(x_S)) \mid x_R \neq x_S] = 2^{-t} \pm \text{negl}(\kappa)$.
- **Coupling.** If $x_R = x_S$ (no tamper), emulate $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ with $b=1$ and, on (ok, sid) , let $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}$ output the honest pair. If $x_R \neq x_S$ (tamper), query the forgery gate with $(\text{Forge}, \text{sid}, m'_S, m'_R)$ in $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$, get b , and prefix-program the RO so equality of the two t -bit SAS values holds iff $b=1$; return the same b from the emulated $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$.

Final distinguishing bound. Let $\Delta(\kappa)$ be the UC advantage between $\text{EXEC}_{Z, \mathcal{A}, \pi_{\text{SAS}}^X}^{\mathcal{F}_{\text{OOB}}^{\text{SAS}}}$ and $\text{IDEAL}_{Z, S, \mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)}$. Then

$$\Delta(\kappa) \leq \mathcal{A}_{\text{Com}}^{\text{hide}}(\kappa) + \mathcal{A}_{\text{Com}}^{\text{bind}}(\kappa) + 2q_H(\kappa)2^{-\kappa} + \left| \Pr_{\text{real}}[\text{accept} \mid x_R \neq x_S] - \varepsilon \right| = \text{negl}(\kappa),$$

since $\Pr_{\text{real}}[\text{accept} \mid x_R \neq x_S] = 2^{-t} \pm \text{negl}(\kappa)$ by the tamper lemma, and $\varepsilon = 2^{-t}$.

Final Simulator (differences vs. prior). The full simulator can be found in Appendix C.

- **Ordering fix.** Matches the real-world message order $C \rightarrow (m_R, r_R) \rightarrow \text{reveal}$; the receiver’s contribution is buffered until the commitment C is observed (no premature receiver start).
- **Other mechanisms unchanged.** Forge-gate invocation (single call on tamper, cached bit b), prefix-constrained lazy RO programming, and ok-coupled delivery remain as in the previous simulator.
- **Attempt-conditioned gate.** The $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}$ forge gate is invoked iff $x_R \neq x_S$.

□

5 Canonical Secure Message Transfer

We now move beyond mere authenticity to confidential and authenticated communication. That is, we aim to realize a standard secure-message-transmission functionality $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ with the addition of a forge-gate attack vector, thus we will use $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ from Fig. 3.4. In the previous section we proved that π_{SAS}^X UC-realizes $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ (Theorem 3). Our first construction leverages π_{SAS}^X to authenticate public-key material (a KEM public key and a digital-signature verification key), after which payloads are sent via a KEM-DEM channel with signed ciphertexts. Our second construction minimizes asymmetric cryptography by replacing digital signatures with a MAC, whose key is contributed and bound through π_{SAS}^X . The second construction is presented in Appendix A. Our target in this section is the canonical realization of $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ of Fig. 3.4. Following Canetti’s recipe [Can20]¹⁰, we combine

¹⁰ See pp. 79 ff. and Claim 27 in the journal version.

- a KEM/DEM channel with a CPA-secure KEM and an OT-CPA-secure DEM for semantic confidentiality,
- an EUF-CMA digital signature to authenticate ciphertexts (and bind sender identity), and
- the cross-authentication functionality $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}$ (Section 3.4) to distribute the KEM public key and the signature verification key.
- a Session identifier sid to denote the SID of this protocol instance. We invoke the subroutine π_{SAS}^X with the same sid (subroutine-respecting).

and obtain the following protocol.

Real Protocol $\pi_{\text{SAS}+\text{KEM}+\text{DEM}+\text{Sig}}$ (hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$)

Primitives and model.

- **KEM** (Gen, Encap, Decap) IND-CPA.
- **DEM** (ENC, DEC) one-time IND-CPA (fresh key per session).
- **Sig** (KGen, Sign, Vfy) EUF-CMA.
- Cross-authentication subroutine π_{SAS}^X in the $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ hybrid.
- Signed context

$$\text{ctx} := \text{"}\pi_{\text{SAS}+\text{KEM}+\text{DEM}+\text{Sig}}\text{"} \parallel \text{sid} \parallel \text{"S} \rightarrow \text{R"}.$$

Parties. Sender S and Receiver R .

Phase 1 — Cross-authentication via π_{SAS}^X .

- 1: S : $(\text{vk}_S, \text{sk}_S) \leftarrow \text{Sig.KGen}$; set $m_S \leftarrow \text{vk}_S$.
- 2: R : $(\text{pk}_R, \text{sk}_R) \leftarrow \text{KEM.Gen}$; set $m_R \leftarrow \text{pk}_R$.
- 3: Run π_{SAS}^X under the *same* sid on inputs (m_S, m_R) .
- 4: Abort iff π_{SAS}^X outputs $b = 0$ (SAS mismatch).

Phase 2 — One-shot KEM–DEM (Enc-Then-Sign).

(2a) *Sender S with payload m :*

- 1: $(K, c_{\text{KEM}}) \leftarrow \text{Encap}(\text{pk}_R)$; $c_{\text{DEM}} \leftarrow \text{DEM.ENC}(K, m)$
- 2: $\sigma \leftarrow \text{Sig.Sign}_{\text{sk}_S}(\text{ctx}, c_{\text{KEM}}, c_{\text{DEM}})$
- 3: $S \rightarrow R$: $(\text{sid}, c_{\text{KEM}}, c_{\text{DEM}}, \sigma)$

(2b) *Receiver R :*

- 4: Abort if $\text{Sig.Vfy}_{\text{vk}_S}(\sigma; \text{ctx}, c_{\text{KEM}}, c_{\text{DEM}}) = 0$
- 5: $K \leftarrow \text{Decap}(\text{sk}_R, c_{\text{KEM}})$; $m \leftarrow \text{DEM.DEC}(K, c_{\text{DEM}})$
- 6: Output (Sent, sid, S , m) to the environment.

Theorem 4 (UC realization of $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ in the $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ -hybrid). Assume:

- (i) KEM is IND-CPA;
- (ii) DEM is one-time OT-CPA;
- (iii) Sig is EUF-CMA;
- (iv) π_{SAS}^X UC-realizes $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ (one online misbinding with success ε).

Protocol $\pi_{SAS+KEM+DEM+Sig}$ is subroutine-respecting (same sid in π_{SAS}^X and in ctx) and uses fresh $(\text{vk}_S, \text{pk}_R)$ per session. Then, in the $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ -hybrid, $\pi_{SAS+KEM+DEM+Sig}$ UC-realizes $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ with length-only leakage. For any PPT environment,

$$\mathcal{A}^{\text{UC}} \leq \varepsilon + \mathcal{A}_{\text{Sig}}^{\text{EUF-CMA}} + \mathcal{A}_{\text{KEM}}^{\text{IND-CPA}} + \mathcal{A}_{\text{DEM}}^{\text{OT-CPA}} + \text{negl}(\kappa).$$

Modular hybrid argument in the $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ -hybrid. Fix a session identifier sid , reused (by convention) inside π_{SAS}^X and included in the signed context ctx . The only allowed leakage to the adversary outside a misbinding success is $|m|$, as required by $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$.

Cross-auth outcome as a single attempt-conditioned bit. In $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ the adversary has *one* online attempt per session; let $B \in \{0, 1\}$ be its success bit (sampled iff an attempt occurs): if $B=0$ both parties receive the honest $(\text{vk}_S, \text{pk}_R)$; if $B=1$ the attacker's substituted keys apply consistently. We condition the data-layer analysis on B .

Authenticity. R accepts iff $\text{Sig}.\text{Vfy}_{\text{vk}_S}(\sigma, (\text{ctx}, c_{\text{KEM}}, c_{\text{DEM}})) = 1$.

If $B=0$, any accepting tuple not created by S is an EUF-CMA forgery on that message:

$$\Pr[\text{accept attacker tuple} \mid B=0] \leq \mathcal{A}_{\text{Sig}}^{\text{EUF-CMA}}.$$

If $B=1$, acceptance under a substituted vk'_S is exactly the single online event accounted for by $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$; no extra loss is charged.

Confidentiality (encrypt-then-sign). Verification precedes decapsulation/decryption; hence the adversary has no decryption oracle.

If $B=0$ (honest keys), use two hybrids:

- H_0 : real,
- H_1 : keep c_{KEM} but replace the DEM key by an independent U ; then
- H_2 : replace $\text{DEM.ENC}(U, m)$ by $\text{DEM.ENC}(U, \mu)$ with $\mu = 0^{|m|}$.

Then

$$|\Pr[H_0 \Rightarrow 1] - \Pr[H_2 \Rightarrow 1]| \leq \mathcal{A}_{\text{KEM}}^{\text{IND-CPA}} + \mathcal{A}_{\text{DEM}}^{\text{OT-CPA}} + \text{negl}(\kappa).$$

If $B=1$ (misbinding), the attacker can redirect encryption and learn m ; $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ mirrors this via its **Reveal** back-door on a successful **Misbind**.

Simulator-ideal coupling (attempt-conditioned). The simulator $\mathcal{S}$ emulates the network and the entire $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}/\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ front-end for the protocol ITIs and couples the one-shot decision to $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$:

- *Trigger.* On back-door $(\text{Sent}, \text{sid}, S, R, |m|)$ from $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$, start the emulated cross-auth under the same sid : generate fresh $(\text{vk}_S, \text{sk}_S)$ and $(\text{pk}_R, \text{sk}_R)$; place C , then $(m_R := \text{pk}_R, r_R)$, then the opening $(m_S := \text{vk}_S, r_S)$, preserving real timing. *Inside the emulated $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$* , use $s = \text{Trunc}_t(H(\cdot))$ when leaking each party's SAS.
- *Coupled gate.* If (and only if) the adversary steers the emulated cross-auth into the tampered branch $(x_R \neq x_S)$, $\mathcal{S}$ *first* locks the single Bernoulli bit by calling

$$(\text{MisbindResult}, \text{sid}, b) \leftarrow \mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon).\text{MisbindTry}(\text{sid}).$$

It then *finishes* the cross-auth emulation using this same b : program the RO so that the two SAS values match iff $b=1$, leak the truncated SAS values to the adversary as $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ would, and return b (accept if 1, abort if 0).

- *Data phase.*
 - If no **Forge** attempt occurred: emulate the honest path; upon (ok, sid) , command $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ to deliver the original m .
 - If a **Forge** attempt occurred and $b=1$: when the attacker injects an accepted ciphertext, use sk_R to decapsulate and extract the delivered payload m' ; upon (ok, sid) , let $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ deliver $(\text{Sent}, \text{sid}, S, m')$ to R , and forward any $(\text{Reveal}, \text{sid}, m)$ back-door to the adversary.
 - If $b=0$: emulate the SAS/signature mismatch (no acceptance); then $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ outputs nothing for this session.

The full simulator can be found at Appendix D.

Conclusion. Combine authenticity (EUF-CMA when $B=0$), confidentiality (KEM IND-CPA + DEM one-time OT-CPA under verify-before-decrypt), and the single attempt-conditioned Bernoulli gate (success probability ε) mirrored by $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$. UC composition yields

$$\mathcal{A}^{\text{UC}} \leq \varepsilon + \mathcal{A}_{\text{Sig}}^{\text{EUF-CMA}} + \mathcal{A}_{\text{KEM}}^{\text{IND-CPA}} + \mathcal{A}_{\text{DEM}}^{\text{OT-CPA}} + \text{negl}(\kappa). \quad (1)$$

which yields the theorem. $\square$

Scope clarification. The present construction is *not* intended as-is as a "bootstrapping procedure" for a multi-message, multi-receiver and multi-sender secure channel in the sense of [SBB⁺23, BGMQ⁺22]. It is a one-shot, single-session-oriented SMT: keys and artifacts are fresh per session and used to deliver a single payload. Critically, every SMT session sid **must** be preceded by a fresh execution of $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ (or its realization π_{SAS}^X) to distribute fresh $(\text{vk}_S, \text{pk}_R)$ that are explicitly bound to sid . Reusing $(\text{vk}_S, \text{pk}_R)$ across sessions undermines authenticity and opens the strip&re-sign and cross-session replay issues discussed below, even if the signature scheme remains EUF-CMA.

If the receiver's KEM public key pk_R and the senders' verification keys vk_S (for various senders) are reused across sessions, the following problem arises: An adversary can take a valid tuple $(c_{\text{KEM}}, c_{\text{DEM}}, \sigma)$ produced in a session with sender S , remove σ , and re-sign $(\text{ctx}', c_{\text{KEM}}, c_{\text{DEM}})$ under some other sender S'

in a different session where R still uses the same pk_R and verifies with $\text{vk}_{S'}$. Since c_{KEM} targets pk_R , R will decapsulate/decrypt and wrongly attribute the payload to S' . This does not violate EUF-CMA for Sig (no forgery under S 's key). It shows that signatures alone do not bind the ciphertexts to the *original* sender if pk_R is reused.

Our protocol avoids these issues by *not* reusing keys: π_{SAS}^X distributes fresh $(\text{vk}_S, \text{pk}_R)$ per session, and ctx includes the session identifier sid . If pk_R or vk_S must be reused (e.g., operational constraints), standard hardenings are e.g. Sender-binding KEMs. Use a sender-binding KEM (IND-SB-CPA) so that decapsulation depends on the sender's identity/public key. Re-signing under S' then fails at decapsulation [SBB⁺23, BGMQ⁺22].

6 Systematization of the Real-World Tooling Landscape

This section surveys how popular tools handle rendezvous, authentication, and transport when two endpoints want to “connect now” with minimal setup. Our goal is to extract the architectural patterns that matter for security and not to audit implementations. We read public docs and code as of 03 Sep 2025 and place each tool along a few common axes: (i) how peers meet (discovery/rendezvous), (ii) what—if anything—the human binds out of band (none / PAKE password / short SAS), (iii) which transport is used (and where encryption terminates), and (iv) what the back-end service learns under an active, malicious-server assumption.

We then look at two slices. First, section 6.1 systematizes one-shot file-transfer tools and contrasts unauthenticated DTLS data channels with PAKE-based “one code” designs, and with our SAS-based alternative. Second, section 6.2 reviews SAS-style checks in deployed systems and separates true *session-bound SAS* from longer, identity-fingerprint comparisons. Throughout, we connect these mechanisms to our modeling $(\mathcal{F}_{\text{OOB}}^{\text{SAS}}, \mathcal{F}_{\text{AUTH}}^{S \rightarrow R} / \mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R})$ to clarify what each pattern actually guarantees when the service operator is adversarial.

6.1 File Transfer in the Real World

We now narrow the scope to one-shot device-to-device transfers between co-located devices owned by the same person. We retain the adversarial-service threat model stated above: signaling/rendezvous infrastructure may be malicious, while on-the-wire transports provide confidentiality and integrity. Under this model the residual question is what the service learns and whether any human out-of-band bind (if any) defeats MitM. We begin with tools that form WebRTC data channels without a user-verifiable bind, then cover PAKE-based “one-code” designs, and finally contrast both with our SAS-based alternative. An asynchronous variant, where comparison occurs later or across users, entails different UX/trust trade-offs and is left for future work.

Unauthenticated DTLS. *PairDrop*¹¹, *ShareDrop*¹², *FilePizza*¹³, and *Snapdrop*¹⁴ establish WebRTC data channels without exposing any user-verifiable binding (no SAS, no Password, no fingerprint UI). Technically, browsers set up an SCTP association over DTLS over UDP. Each peer’s DTLS certificate fingerprint is conveyed inside SDP via the signaling channel (see WebRTC security architecture RFC 8826/8827). If signaling is malicious, it can substitute SDPs and fingerprints and terminate two DTLS associations, achieving a full MitM despite DTLS on the wire. This is not a flaw in DTLS or SCTP but a missing *identity bind*: the browser verifies that the DTLS cert matches the fingerprint it received, but there is no human check that this fingerprint corresponds to the intended peer. In other words, these tools perform an unauthenticated Diffie-Hellman exchange.

A clean mitigation is to add a user-verifiable bind. Our framework provides exactly that: a cross-authentication functionality $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ realized from a short-authentication-string (SAS) over an authenticated OOB channel. Concretely, one can display a short fingerprint/SAS derived from the agreed transcript and ask both sides to compare it over an auxiliary authenticated channel and if it matches, the session is user-bound and MitM is defeated under the stated $\varepsilon = 2^{-t}$ budget. The *noisytransfer* protocol implementation¹⁵ and the companion *noisyauth* npm package¹⁶ are available for this purpose. It is also worthwhile mentioning that at *PairDrop* it is already being discussed about adding similar protections¹⁷.

PAKE-based. Tools like *Magic Wormhole*¹⁸, *WebWormhole*¹⁹, and *croc*²⁰ follow the same architectural pattern. The sender locally creates a single human-readable code and shares it out of band with the receiver. Codes have usually the form *N-word-word...* (e.g., *7-crossover-clockwork*). That code combines two roles: a rendezvous reference (e.g., a mailbox nameplate or a server slot) and a PAKE password. The receiver inputs the code, which lets both sides attach to the same rendezvous and then run a PAKE to authenticate and derive keys. Two integration styles are common: (i) PAKE directly yields application-layer channel keys for encrypted streaming (e.g., SPAKE2 [Lad23] in *Magic Wormhole* and *croc*), or (ii) PAKE authenticates²¹ the signaling metadata before the data channel forms (e.g., CPace protects the SDP carrying DTLS certificate fingerprints in *WebWormhole* [HO25]). Under a malicious-server assumption, the

¹¹ <https://github.com/schlagmichdoch/PairDrop>

¹² <https://github.com/ShareDropio/sharedrop>

¹³ <https://github.com/kern/filepizza>

¹⁴ <https://github.com/SnapDrop/snapdrop>

¹⁵ <https://github.com/collapsinghierarchy/noisytransfer-protocol>

¹⁶ <https://www.npmjs.com/package/@noisytransfer/noisyauth>

¹⁷ <https://github.com/schlagmichdoch/PairDrop/issues/180>

¹⁸ <https://github.com/magic-wormhole/magic-wormhole>

¹⁹ <https://github.com/saljam/webwormhole>

²⁰ <https://github.com/schollz/croc>

²¹ and unnecessarily encrypts

service learns the rendezvous identifier and sees the PAKE transcript but not the password. Security then hinges on online-guess resistance and careful client-side handling. This pattern delivers a strong user experience: one code, typed once. It also defines a practical ceiling for PAKE-based UX because the out-of-band message must carry a sufficiently long shared secret alongside rendezvous data.

Our SAS-based approach decouples these concerns: use a short, non-secret rendezvous code purely for meeting, then display and compare a short SAS for binding. No long secret needs to be carried out of band, and the residual misbinding risk is explicit via the $\varepsilon = 2^{-t}$ budget, assuming a synchronous $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ is available as in our model. It is an interesting open question which of the paradigms provide the best usability/security tradeoff.

6.2 SAS Handling in the Real World

Beyond file transfer, many deployed systems ask users to compare short strings or longer fingerprints to detect MitM. The items below use *identity-key fingerprints* (often with a QR fallback), not a short, session-bound SAS in the formal sense. They are excellent for long-term identity verification in persistent conversations or ecosystems, but they do not authenticate a fresh one-shot transcript and their displayed values are much larger (e.g., 60 decimal digits) and change only when long-term keys rotate. We therefore exclude them from our SAS-focused analysis:

- **Signal.** Each 1:1 chat shows a safety number (numeric and QR) that fingerprints the parties’ identity keys; when keys change, the app forces re-verification [Sig24]. This is an identity fingerprint, not a per-session SAS.
- **WhatsApp.** Each chat exposes a security code (60 digits and QR) that users compare/scan to verify the chat’s long-term keys [Wha24]; again a fingerprint rather than a live SAS.
- **Apple iMessage (Contact Key Verification).** Users compare short verification codes (or rely on posted public verification codes) to detect key substitution on Apple’s servers; the check binds directory-backed identity keys, not a transient handshake [App23a, App23b, App24].
- **Wire.** Conversations/devices expose key fingerprints (with QR) for out-of-band comparison, targeting identity/device trust establishment rather than per-run SAS binding [Wir25, Wir24].
- **Threema.** Contacts are verified by scanning a QR code or comparing the public-key fingerprint to prevent MitM; this is a stable identity fingerprint, not recomputed per session [Thr25b, Thr25a].
- **Telegram Secret Chats.** Clients display a key visualization derived from the chat’s keys for users to compare; functionally this is a human-verifiable fingerprint for the chat, not a short SAS of a fresh run [Tel25].

ZRTP SAS derivation. ZRTP derives a human-verifiable SAS during its DH key agreement ([ZJC11], §4–§7).

- **Handshake setup.** After Hello/HelloAck ([ZJC11], §4–§7), the initiator A fixes its DH payload by sending Commit with a hash commitment $\text{hvi} = \text{Hash}(\text{DHPart2}_A \parallel \text{Hello}_B)$.

- **DH exchange and check.** Peers exchange DHPart1/DHPart2; verify $\text{hvi} \stackrel{?}{=} \text{Hash}(\text{DHPart2}_A \parallel \text{Hello}_B)$. Compute the shared DH value and derive the master secret s_0 .
- **Context for SAS.** Set $\text{KDF_Context} := \text{ZID}_i \parallel \text{ZID}_r \parallel \text{total_hash}$, where ZID_* are the 96-bit ZRTP identifiers and total_hash is the running hash of all ZRTP messages in this run.
- **Derive SAS bits.** Using the spec’s HMAC-based KDF (NIST SP 800-108 style; not necessarily HKDF), compute

$$\text{sashash} := \text{KDF}(s_0, \text{"SAS"}, \text{KDF_Context}, 256)$$

and take $v := \text{MSB}_{32}(\text{sashash})$.

- **Render to users.** If B32 was negotiated, show the MSB 20 bits of v as four Base32 characters; if B256, show the MSB 16 bits as two PGP-word-list words ([ZJC11], §5.1.6). Users compare the display verbally/visually.

This is a classical SAS-derivation protocol in the Vaudenay/MANA sense: the commitment caps active attacker adaptivity, the SAS is bound to the live transcript via KDF_Context , and a successful MitM requires guessing the t -bit display. In our terminology, ZRTP aligns with π_{SAS}^X when we view the protocol messages as $m_S := \text{DHPart2}_A$ and $m_R := \text{DHPart1}_B$, and the verbal/visual code comparison as the $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ step. If the used KDF is modeled as a Random Oracle the universal composability from π_{SAS}^X thus also translates to ZRTP SAS-derivation.

Matrix/Element: Matrix implements a SAS verification (`m.sas.v1`) that follows the same Vaudenay/MANA commit, reveal, compare pattern as ZRTP: the acceptor first commits to its ephemeral key (hash over key and other information), both sides run ECDH, and the SAS is derived via HKDF-SHA256 from the shared secret and an info string binding user IDs, device IDs, both ephemeral keys, and a transaction id, then rendered as three 4-digit numbers or seven emoji [Mat24]. After users confirm a match, Matrix additionally MAC-authenticates each device’s long-term keys under a key from the same ECDH output, tying the human check to concrete device identities. At a high level this is the same session-bound SAS approach as ZRTP/MANA-IV/MA-DH. The small differences are the commitment target and the identity-binding step that follows the SAS.

Bluetooth LE Secure Connections (Numeric Comparison) SAS. In LE Secure Connections, both sides exchange ECDH public keys (PK_A, PK_B), commit to fresh nonces via **Pairing Confirm**, reveal the nonces via **Pairing Random**, and then display a 6-digit value for human comparison (the “SAS”). This follows the MANA-IV commit, reveal and compare pattern, with the difference that both endpoints commit (each sends a confirm) rather than only the initiator.

Concretely, the commitment values and the final SAS are:

$$\begin{aligned} C_A &= f4(\text{PK}_{Ax}, \text{PK}_{Bx}, N_A, 0) \\ C_B &= f4(\text{PK}_{Bx}, \text{PK}_{Ax}, N_B, 0) \\ \text{SAS} &= g2(\text{PK}_{Ax}, \text{PK}_{Bx}, N_A, N_B) \bmod 10^6 \end{aligned}$$

Where $f4$ and $g2$ are defined as AES-CMAC keyed by the (pre-reveal and thus still-secret) nonce N_A , with $g2$ first reduced $\bmod 2^{32}$ before rendering as six decimal digits. Security thus hinges on two CMAC-based components: (i) $f4$ as a computational “commitment” to the nonce bound to both public keys, and (ii) $g2$ as a PRF-derived short value over $(\text{PK}_{Ax}, \text{PK}_{Bx}, N_A, N_B)$. Under standard PRF/MAC assumptions for AES-CMAC and fresh, uniform nonces, $f4$ is *hiding* (the tag leaks nothing about the nonce prior to reveal) and *binding* (after sending C on a fixed input, opening to a different nonce would require finding a new CMAC key that reproduces C), and $g2$ yields a value computationally indistinguishable from uniform on $\{0, 1\}^{32}$ (hence $\approx 10^{-6}$ success after decimal reduction).

However, unlike our π_{SAS}^X protocol, BLE’s $g2$ is not modeled as a random oracle. Our UC simulator for π_{SAS}^X relies on a programmable random oracle to (A) guarantee exact uniformity under truncation and (B) program equal/different t -bit prefixes at the forge-gate. A fixed PRF such as CMAC does not provide that programmability. Thus, while BLE’s Numeric Comparison is sound in the usual game-based sense, the UC simulation strategy we use for π_{SAS}^X does not carry over verbatim to BLE’s SAS derivation. In Appendix F we sketch an RO-free alternative (coin-flip c and commit-then-bind) that attains the same user interface while allowing a clean UC proof with standard UC commitments and a UC coin-flip ([CF01, CLOS02, Blu83]). It is, however, conceptually different from the BLE handshake.

7 Conclusion

We studied instant, one-shot device-to-device exchanges under an active network adversary without PKI or pre-shared secrets. Our main result is that a short, human-verifiable comparison can be treated as a first-class resource with a precise, composable failure budget $\varepsilon = 2^{-t}$. We modeled this via $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$, gave simple commitment-style protocols π_{SAS} and π_{SAS}^X that are UC equivalents of MANA IV, proved they realize $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ and $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$, and showed how to compose them with standard KEM/DEM into $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ while preserving the explicit ε . Practically, this decouples rendezvous from authentication: the out-of-band step shrinks to comparing a short t -bit string instead of transporting a long shared secret.

Our systematization clarifies where today’s tools stand. Browser WebRTC data channels commonly lack a user-verifiable bind and are thus vulnerable to adversarial signaling. PAKE “one-code” designs deliver strong usability but couple rendezvous codes with out-of-band secrets. Session-bound SAS fixes the missing bind with an explicit risk budget (identical to PAKEs) and can be retrofitted

to existing stacks by hashing the negotiated transcript and showing a short code to both sides. We released reference code²² to make this easy to integrate.

Limitations and future work. Our mainline proofs use a random-oracle instantiation for hashing and commitments. We sketched an RO-free variant based on a UC coin-flip and non-malleable commitments, but the full proof is left for future work. We assumed synchronous, co-present users and set the human-error parameter $\varepsilon_{\text{human}} = 0$ to isolate cryptographic guarantees. A calibrated treatment of human fallibility and asynchronous comparisons is also left to future work. Additional directions include group extensions, a stronger treatment of compromised UIs and denial-of-service (DoS) against the OOB channel, and analyzing SAS-based protocols within a deniability framework.

Acknowledgements

We would like to thank Felix Dörre and Laurin Benz for their early Feedback.

References

- And24. Android Open Source Project. Wi-fi easy connect (device provisioning protocol, dpp). <https://source.android.com/docs/core/connect/wifi-easy-connect>, 2024. Accessed 2025-09-03.
- App23a. About imessage contact key verification. <https://support.apple.com/en-us/118246>, 2023.
- App23b. Advancing imessage security: Contact key verification. <https://security.apple.com/blog/imessage-contact-key-verification/>, 2023.
- App24. Use contact key verification on iphone. <https://support.apple.com/guide/iphone/use-contact-key-verification-iph654dd8c53/ios>, 2024.
- BGMQ⁺22. Wasilij Beskorovajnov, Roland Gröll, Jörn Müller-Quade, Astrid Ottenhues, and Rebecca Schwerdt. A new security notion for pkc in the standard model: weaker, simpler, and still realizing secure channels. In *IACR International Conference on Public-Key Cryptography*, pages 316–344. Springer, 2022.
- Blu83. Manuel Blum. Coin flipping by telephone. In *CRYPTO 1981 (reprinted in SIGACT News, 1983)*, 1983. Commit-then-open coin tossing.
- BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *ACM CCS*, pages 62–73, 1993.
- BT15. Almog Benin, Sivan Toledo, and Eran Tromer. Secure association for the internet of things. In *2015 International Workshop on Secure Internet of Things (SIoT)*, pages 25–34. IEEE, 2015.
- Can01. Ran Canetti. Universally composable security: a new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145, oct 2001.

²² <https://github.com/collapsinghierarchy/noisytransfer-protocol>

- Can20. Ran Canetti. Universally composable security. *Journal of the ACM (JACM)*, 67(5):1–94, 2020.
- CF01. Ran Canetti and Marc Fischlin. Universally composable commitments. In *CRYPTO 2001*. Springer, 2001. UC commitments in CRS; enables UC coin-toss via $\text{commit} \oplus$.
- CLOS02. Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai. Universally composable two-party and multi-party secure computation. In *STOC 2002*, 2002. General UC framework; CRS-hybrid feasibility and building blocks.
- Exc05. Key Exchange. Universally composable password-based. In *Advances in Cryptology-EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005, Proceedings*, volume 3494, page 404. Springer Science & Business Media, 2005.
- FÁS⁺18. Mikhail Fomichev, Flor Álvarez, Daniel Steinmetzer, Paul Gardner-Stephen, and Matthias Hollick. Survey and systematization of secure device pairing. *IEEE Communications Surveys & Tutorials*, 20(1):517–550, 2018. Preprint: <https://arxiv.org/abs/1709.02690>.
- GMS10. Ian Goldberg, Atefeh Mashatan, and Douglas R. Stinson. On message recognition protocols: Recoverability and explicit confirmation. *International Journal of Applied Cryptography*, 2(4):331–346, 2010.
- HO25. Björn Haase and Others. Cpace, a balanced composable pake. <https://datatracker.ietf.org/doc/draft-irtf-cfrg-cpace/>, 2025. Internet-Draft, IRTF CFRG.
- HWGW05. Jonathan Hammell, André Weimerskirch, João Girão, and Dirk Westhoff. Recognition in a low-power environment. In *Proceedings of the 25th IEEE International Conference on Distributed Computing Systems Workshops (ICDCSW'05) — Second International Workshop on Wireless Ad Hoc Networking (WWAN)*, pages 933–938, Columbus, OH, USA, 2005. IEEE Computer Society.
- JS10. Stanisław Jarecki and Nitesh Saxena. Authenticated key agreement with key re-use in the short authenticated strings model. In *International Conference on Security and Cryptography for Networks*, volume 6280 of *LNCS*, pages 253–270. Springer, Springer, 2010.
- KL20. Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 3rd edition, 2020.
- KST⁺09. Alfred Kobsa, Rahim Sonawalla, Gene Tsudik, Ersin Uzun, and Yang Wang. Serial hook-ups: A comparative usability study of secure device pairing methods. In *Proceedings of the 5th Symposium on Usable Privacy and Security (SOUPS 2009)*, Mountain View, CA, USA, 2009. ACM. Author copy: <https://yangwang.ischool.illinois.edu/papers/SOUPS09-AuthorCopy.pdf>.
- KSTU09. Arun Kumar, Nitesh Saxena, Gene Tsudik, and Ersin Uzun. A comparative study of secure device pairing methods. *Pervasive and Mobile Computing*, 5(6):734–749, 2009.
- Lad23. Watson Ladd. Spake2, a password-authenticated key exchange. <https://www.rfc-editor.org/rfc/rfc9382>, September 2023. IRTF RFC 9382.
- Lin11. Yehuda Lindell. A simpler construction of universally composable commitments using trapdoor permutations. In *TCC 2011*. Springer, 2011.

- LN06a. Sven Laur and Kaisa Nyberg. Efficient mutual data authentication using manually authenticated strings. In *Cryptology and Network Security: 5th International Conference, CANS 2006, Suzhou, China, December 8-10, 2006, Proceedings*, volume 4301, pages 90–107. Springer, 2006.
- LN06b. Sven Laur and Kaisa Nyberg. Efficient mutual data authentication using manually authenticated strings (extended version). Technical Report 2005/424, IACR ePrint, 2006.
- LP08. Sven Laur and Sylvain Pasini. Sas-based group authentication and key agreement protocols. In *PKC 2008*, volume 4939 of *LNCS*, pages 197–213. Springer, 2008.
- LP09. Sven Laur and Sylvain Pasini. User-aided data authentication. *International Journal of Security and Networks*, 4(1-2):69–86, 2009.
- Mat24. Matrix.org. Implementing more advanced e2ee features (sas emoji/numbers). <https://matrix.org/docs/guides/implementing-more-advanced-e-2-ee-features-such-as-cross-signing/>, 2024. Accessed 2025-09-03.
- MPR05. Jonathan M. McCune, Adrian Perrig, and Michael K. Reiter. Seeing-is-believing: Using camera phones for human-verifiable authentication. In *Proceedings of the 2005 IEEE Symposium on Security and Privacy (S&P)*, pages 110–124. IEEE Computer Society, 2005.
- MV10. Atefeh Mashatan and Serge Vaudenay. A message recognition protocol based on standard assumptions. In *ACNS 2010*, volume 6123 of *LNCS*, pages 384–401. Springer, 2010.
- PR⁺21. Max Pritikin, Michael Richardson, et al. Bootstrapping remote secure key infrastructure (brski). RFC 8995, 2021.
- PV06a. Sylvain Pasini and Serge Vaudenay. An optimal non-interactive message authentication protocol. In *CT-RSA 2006*, volume 3860 of *LNCS*, pages 280–296. Springer, 2006.
- PV06b. Sylvain Pasini and Serge Vaudenay. Sas-based authenticated key agreement. In *International Workshop on Public Key Cryptography*, pages 395–409. Springer, 2006.
- SBB⁺23. Rebecca Schwerdt, Laurin Benz, Wasilij Beskorovajnov, Sarai Eilebrecht, Jörn Müller-Quade, and Astrid Ottenhues. Sender-binding key encapsulation. *Cryptology ePrint Archive*, 2023.
- Sig24. Signal Messenger. What is a safety number and why do i see that it changed? <https://support.signal.org/hc/en-us/articles/360007060632>, 2024. Accessed 2025-09-03.
- Tel25. Telegram secret chats: End-to-end encryption. <https://core.telegram.org/api/end-to-end>, 2025. Key visualization and fingerprint details.
- Thr25a. Threema support: Private faq (identity verification). <https://threema.com/en/support/private>, 2025.
- Thr25b. What is the qr code good for? <https://threema.com/en/faq/qr-code-expl>, 2025.
- Vau05. Serge Vaudenay. Secure communications over insecure channels based on short authenticated strings. In *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 309–326. Springer-Verlag, 2005.
- Wha24. WhatsApp. About security code change notifications. <https://faq.whatsapp.com/1524220618005378>, 2024. Accessed 2025-09-03.

- Wir24. How can i compare key fingerprints? <https://support.wire.com/hc/en-us/articles/207692235-How-can-I-compare-key-fingerprints>, 2024.
- Wir25. Wire security & privacy. <https://wire.com/en/security>, 2025.
- ZJC11. Philip R. Zimmermann, Alan Johnston, and Jon Callas. ZRTP: Media path key agreement for unicast secure RTP. RFC 6189, 2011.

A MAC-based SMT

Our next goal is to reduce the asymmetric footprint of the SMT stack by replacing the signature layer with symmetric authentication, while keeping the SAS-based cross-authentication as the only out-of-band (OOB) trust anchor. The main motivation is efficiency: asymmetric primitives are typically more costly than symmetric ones, as argued in [BTT15]. This follows a common design line in SAS/PAKE-backed channels, where public-key signing can be traded for MACs keyed by ephemeral session material, but at the price of losing public verifiability and the “signature firewall” (i.e., verification before any decryption/long-term use). We note that the loss of public verifiability may, in some settings, support deniability goals, however a formal deniability analysis of our construction is out of scope. We retain the UC modelling of SAS via $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ and cross-authentication via π_{SAS}^X , and we continue to target the same ideal functionality $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ with length-only leakage and a single online misbinding event of success probability $\varepsilon = 2^{-t}$.

Real Protocol $\pi_{\text{SAS}+\text{KEM}+\text{DEM}+\text{MAC}}$ (hybrid $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$)

Primitives.

- Equivocable commitment $\text{Com} = (\text{Commit}, \text{Open})$: hiding, binding, with trapdoor for equivocation (used only by the UC simulator).
- KEM $\text{KEM} = (\text{Gen}, \text{Encap}, \text{Decap})$: IND-CPA.
- DEM $\text{DEM} = (\text{ENC}, \text{DEC})$: *one-time* IND-CPA (fresh KEM key per session).
- MAC (e.g. HMAC) with UF-CMA security.
- Random oracle H for SAS; $\text{sas}(x) = \text{Trunc}_t(H(x))$.

Parties. Sender S sends a single message m to receiver R .

Context string.

$$\text{ctx} := \text{“}\pi_{\text{SAS}+\text{KEM}+\text{DEM}+\text{MAC}}\text{”} \parallel \text{sid} \parallel \text{“}S \rightarrow R\text{”}.$$

Phase 1 — Cross-authentication via π_{SAS}^X (SID reuse)

- 1: S : sample $K_{\text{MAC}} \leftarrow \$ \{0,1\}^\kappa$ and commitment coins r_S ; set $C_S \leftarrow \text{Commit}(K_{\text{MAC}}; r_S)$; define $m_S \leftarrow C_S$.
- 2: R : $(\text{pk}_R, \text{sk}_R) \leftarrow \text{KEM.Gen}$; sample $r_R \leftarrow \$ \{0,1\}^\kappa$; define $m_R \leftarrow \text{pk}_R$.
- 3: Run π_{SAS}^X under the *same* sid with inputs (m_S, m_R) .
- 4: **Abort** iff π_{SAS}^X outputs $(\text{SAS}, \text{sid}, 0)$.
- 5: (On success $b=1$: S learns (pk_R, r_R) and R learns C_S ; SAS leakage flows via $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ as usual.)

Phase 2 — One-shot transport with post-decrypt MAC check

- (2a) *Sender S with payload m:*
- 1: $(K, c_{\text{KEM}}) \leftarrow \text{KEM.Encap}(\text{pk}_R)$
 - 2: $P \leftarrow (m, K_{\text{MAC}}, r_S)$ (carry MAC key & its opening inside the DEM payload)
 - 3: $c_{\text{DEM}} \leftarrow \text{DEM.ENC}(K, P)$
 - 4: $\tau \leftarrow \text{MAC}_{K_{\text{MAC}}}(\text{ctx} \parallel c_{\text{KEM}} \parallel c_{\text{DEM}})$
 - 5: $S \rightarrow R : (\text{sid}, c_{\text{KEM}}, c_{\text{DEM}}, \tau)$
- (2b) *Receiver R:*
- 6: $K \leftarrow \text{KEM.Decap}(\text{sk}_R, c_{\text{KEM}})$ (abort on failure)
 - 7: $P \leftarrow \text{DEM.DEC}(K, c_{\text{DEM}})$; parse P as (m, K_{MAC}, r_S) (abort on failure)
 - 8: **Commitment check:** accept only if $\text{Open}(C_S; r_S, K_{\text{MAC}})$ succeeds
 - 9: **Tag check:** accept only if $\text{MAC}_{K_{\text{MAC}}}(\text{ctx} \parallel c_{\text{KEM}} \parallel c_{\text{DEM}}) = \tau$
 - 10: If both checks pass, output $(\text{Sent}, \text{sid}, S, m)$ to the environment; else abort.

We leave the proof for future work.

B Full Simulator of π_{SAS} Coupled with $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\epsilon)$ **Final Simulator $\mathcal{S}$**

Global. Lazy random oracle $T : \{0, 1\}^* \rightarrow \{0, 1\}^n$ (initially empty). On query x , return $T[x]$ if set; else sample $y \leftarrow U(\{0, 1\}^n)$, set $T[x] \leftarrow y$, return y .

Per-session state (sid).

- $$m_S, C, r_S, r_R, \tilde{r}_R, m'_S, r'_S \leftarrow \perp;$$
- $$\text{attempted} \leftarrow \text{false}; \quad b \leftarrow \perp; \quad \text{forged} \leftarrow \perp.$$
- 1: **On back-door** $(\text{Sent}, \text{sid}, S, m_S)$ **from** $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\epsilon)$:
 - 2: sample $C \leftarrow \$ \{0, 1\}^{n(\kappa)}$; relay C on $S \rightarrow R$
 - 3: sample $r_R \leftarrow \$ \{0, 1\}^\kappa$; later (under $\mathcal{A}$'s scheduling) relay r_R on $R \rightarrow S$
 - 4: set $\text{attempted} \leftarrow \text{false}$, $b \leftarrow \perp$, $\text{forged} \leftarrow \perp$
 - 5: **On seeing a commitment** C' **on** $S \rightarrow R$: (record only; tamper is detected via $x_R \neq x_S$)
 - 6: **On seeing an opening** (m'_S, r'_S) **on** $S \rightarrow R$: store (m'_S, r'_S) .
 - 7: **On seeing** $\tilde{r}_R$ **on** $R \rightarrow S$: store $\tilde{r}_R$.
 - 8: **At SAS comparison:**
 - 9: form $x_R \leftarrow (\text{sid} \parallel m'_S \parallel r'_S \parallel r_R)$
 - 10: form $x_S \leftarrow (\text{sid} \parallel m_S \parallel r_S \parallel \tilde{r}_R)$
 - 11: **if** $x_R = x_S$ **then** $\triangleright$ honest path
 - 12: **if** $x_R \notin T$ **then**
 - 13: sample $y \leftarrow \$ \{0, 1\}^n$; set $T[x_R] \leftarrow y$
 - 14: **end if**
 - 15: $s \leftarrow \text{Trunc}_t(T[x_R])$
 - 16: emulate $\mathcal{F}_{\text{OOb}}^{\text{SAS}}$ (both parties submit s ; SAS values leaked within the emulation); return $b=1$

Final Simulator $\mathcal{S}$ (cont.)

```

17: else  $\triangleright$  tamper detected  $\rightarrow$  one-shot forge gate (attempted/cached-bit pattern)
18:   if  $\neg$ attempted then
19:     send (Forge, sid,  $m'_S, R$ ) to  $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ ; receive (ForgeResult, sid,  $b$ )
20:     set attempted  $\leftarrow$  true; forged  $\leftarrow b$ 
21:   end if
22:   if forged == 1 then  $\triangleright$  equal  $t$ -bit prefixes on distinct inputs
23:     if  $x_R \notin T$  and  $x_S \notin T$  then
24:       sample  $p \leftarrow \$ \{0, 1\}^t$  and  $u, v \leftarrow \$ \{0, 1\}^{n-t}$ ; set  $T[x_R] \leftarrow p \| u$ ,
        $T[x_S] \leftarrow p \| v$ 
25:     else if  $(x_R \in T) \text{ xor } (x_S \in T)$  then
26:       w.l.o.g.  $x_R \in T$ ; sample  $v \leftarrow \$ \{0, 1\}^{n-t}$ ; set  $T[x_S] \leftarrow$ 
        $\text{Trunc}_t(T[x_R]) \| v$ 
27:     else
28:       halt (would contradict negligible pre-hit)
29:     end if
30:   else  $\triangleright$  different  $t$ -bit prefixes
31:     if  $x_R \notin T$  and  $x_S \notin T$  then
32:       sample distinct  $p_R, p_S \leftarrow \$ \{0, 1\}^t$  and  $u, v \leftarrow \$ \{0, 1\}^{n-t}$ ; set
        $T[x_R] \leftarrow p_R \| u$ ,  $T[x_S] \leftarrow p_S \| v$ 
33:     else if  $(x_R \in T) \text{ xor } (x_S \in T)$  then
34:       w.l.o.g.  $x_R \in T$ ; choose  $p_S \neq \text{Trunc}_t(T[x_R])$ ; sample  $v$ ; set
        $T[x_S] \leftarrow p_S \| v$ 
35:     else
36:       halt (would contradict negligible pre-hit)
37:     end if
38:   end if
39:   let  $s_R \leftarrow \text{Trunc}_t(T[x_R])$  and  $s_S \leftarrow \text{Trunc}_t(T[x_S])$ 
40:   emulate  $\mathcal{F}_{\text{OAB}}^{\text{SAS}}$  with (SAS, sid,  $s_R$ ) from  $R$  and (SAS, sid,  $s_S$ ) from  $S$ .
41: end if

42: On (ok, sid) from  $\mathcal{A}$ :
43:   forward (ok, sid) to  $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$  unconditionally.
44:   (By the tri-state logic of  $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}(\varepsilon)$ : no attempt  $\Rightarrow$  honest delivery; attempt
    $\mathcal{E}$  success  $\Rightarrow$  forged delivery; attempt  $\mathcal{E}$  fail  $\Rightarrow$  no delivery/abort.)

```

C Full Simulator of π_{SAS}^X Coupled with $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ **Final Simulator $\mathcal{S}$**

Random Oracle. Lazy random oracle $T : \{0, 1\}^* \rightarrow \{0, 1\}^n$ (initially empty). On query x , return $T[x]$ if set; otherwise sample $y \leftarrow U(\{0, 1\}^n)$, set $T[x] \leftarrow y$, and return y .

Per-session state (sid).

$m_S, m_R \leftarrow \perp$; $C \leftarrow \perp$; $r_S, r_R, \tilde{r}_R \leftarrow \perp$; $\tilde{m}_R \leftarrow \perp$;
 $m'_S, r'_S \leftarrow \perp$; attempted $\leftarrow$ false; $b \leftarrow \perp$; forged $\leftarrow \perp$;
 sentC, sentRmsg, sentOpen $\in \{\text{false}, \text{true}\}$ (all init false); pendingR $\leftarrow$ false.

Final Simulator $\mathcal{S}$ (cont.)

```

1: On back-door (Seen, sid,  $S, m_S$ ) from  $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ :
2:   store  $m_S$ ; sample  $r_S \leftarrow U(\{0, 1\}^\kappa)$ ,  $C \leftarrow U(\{0, 1\}^{n(\kappa)})$ ; place  $C$  on  $S \rightarrow R$ ;
   set sentC  $\leftarrow$  true
3: if pendingR and  $\neg$ sentRmsg then
4:   sample  $r_R \leftarrow U(\{0, 1\}^\kappa)$ ; place  $(m_R, r_R)$  on  $R \rightarrow S$ ; set sentRmsg  $\leftarrow$  true
5:   place the opening  $(m_S, r_S)$  on  $S \rightarrow R$ ; set sentOpen  $\leftarrow$  true
6: end if

7: On back-door (Seen, sid,  $R, m_R$ ) from  $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ :
8:   store  $m_R$ 
9: if sentC and  $\neg$ sentRmsg then
10:  sample  $r_R \leftarrow U(\{0, 1\}^\kappa)$ ; place  $(m_R, r_R)$  on  $R \rightarrow S$ ; set sentRmsg  $\leftarrow$  true
11:  place the opening  $(m_S, r_S)$  on  $S \rightarrow R$ ; set sentOpen  $\leftarrow$  true
12: else
13:   set pendingR  $\leftarrow$  true
14: end if

15: On seeing an opening  $(m'_S, r'_S)$  on  $S \rightarrow R$ : store  $(m'_S, r'_S)$ .
16: On seeing  $(\tilde{m}_R, \tilde{r}_R)$  on  $R \rightarrow S$ : store  $(\tilde{m}_R, \tilde{r}_R)$ .

17: At SAS comparison:
18:    $x_R \leftarrow (\text{sid} \parallel m'_S \parallel m_R \parallel r'_S \parallel r_R)$ 
19:    $x_S \leftarrow (\text{sid} \parallel m_S \parallel \tilde{m}_R \parallel r_S \parallel \tilde{r}_R)$ 
20: if  $x_R = x_S$  then  $\triangleright$  honest path
21:   if  $x_R \notin T$  then
22:     sample  $y \leftarrow U(\{0, 1\}^n)$ ; set  $T[x_R] \leftarrow y$ 
23:   end if
24:    $s \leftarrow \text{Trunc}_t(T[x_R])$ 
25:   emulate  $\mathcal{F}_{\text{OoB}}^{\text{SAS}}$  (both parties submit  $s$ ; leak SAS per-party); return  $b=1$ 
26: else  $\triangleright$  tamper detected  $\rightarrow$  one-shot forge gate (attempted/cached-b pattern)
27:   if  $\neg$ attempted then
28:     send (Forge, sid,  $m'_S, \tilde{m}_R$ ) to  $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ ; receive (ForgeResult, sid,  $b$ )
29:     set attempted  $\leftarrow$  true; forged  $\leftarrow b$ 
30:   end if
31:   if forged == 1 then  $\triangleright$  equal  $t$ -bit prefixes on distinct inputs
32:     if  $x_R \notin T$  and  $x_S \notin T$  then
33:       sample  $p \leftarrow U(\{0, 1\}^t)$ ,  $u, v \leftarrow U(\{0, 1\}^{n-t})$ ; set  $T[x_R] \leftarrow p \parallel u$ ,
        $T[x_S] \leftarrow p \parallel v$ 
34:     else if  $(x_R \in T) \text{ xor } (x_S \in T)$  then
35:       w.l.o.g.  $x_R \in T$ ; sample  $v \leftarrow U(\{0, 1\}^{n-t})$ ; set  $T[x_S] \leftarrow$ 
        $\text{Trunc}_t(T[x_R]) \parallel v$ 
36:     else
37:       halt (would contradict negligible pre-hit)
38:     end if
39:   else  $\triangleright$  different  $t$ -bit prefixes
40:     if  $x_R \notin T$  and  $x_S \notin T$  then

```

Final Simulator $\mathcal{S}$ (cont.)

```

41:      sample distinct  $p_R, p_S \in \{0, 1\}^t$  and  $u, v \leftarrow U(\{0, 1\}^{n-t})$ ; set
       $T[x_R] \leftarrow p_R \| u, T[x_S] \leftarrow p_S \| v$ 
42:      else if  $(x_R \in T) \text{ xor } (x_S \in T)$  then
43:          w.l.o.g.  $x_R \in T$ ; choose  $p_S \neq \text{Trunc}_t(T[x_R])$ ; sample  $v$ ; set
           $T[x_S] \leftarrow p_S \| v$ 
44:      else
45:          halt (would contradict negligible pre-hit)
46:      end if
47:  end if
48:      let  $s_R \leftarrow \text{Trunc}_t(T[x_R])$  and  $s_S \leftarrow \text{Trunc}_t(T[x_S])$ 
49:      emulate  $\mathcal{F}_{\text{OoB}}^{\text{SAS}}$  with  $(\text{SAS}, \text{sid}, s_R)$  from  $R$  and  $(\text{SAS}, \text{sid}, s_S)$  from  $S$ .
50: end if

51: On (ok, sid) from  $\mathcal{A}$ :
52:   forward (ok, sid) to  $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$  unconditionally.
53:   (By the tri-state logic of  $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ : no attempt  $\Rightarrow$  honest delivery; attempt
   & success  $\Rightarrow$  forged delivery; attempt & fail  $\Rightarrow$  no delivery/abort.)

```

D Full Simulator of $\pi_{\text{SAS}+\text{KEM}+\text{MAC}+\text{Sig}}$ Coupled with $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ in the $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$ Hybrid Model

Final Simulator $\mathcal{S}$ (cross-auth $\leftrightarrow$ MitM coupling)

Per-session state (sid).

```

 $\text{vk}_S, \text{sk}_S, \text{pk}_R, \text{sk}_R, \text{m}' \leftarrow \perp$ ;    $\text{sentC}, \text{sentResp}, \text{sentOpen} \in \{\text{false}, \text{true}\}$  (init false);
 $\text{attempted} \in \{\text{false}, \text{true}\}$  (init false);    $\text{forged} \in \{\perp, \text{true}, \text{false}\}$  (init  $\perp$ );
 $\text{mprime} \leftarrow \perp$ ;    $\text{delivered} \leftarrow \text{false}, \text{origEmitted} \leftarrow \text{false}; \text{vkToR} \leftarrow \perp$ ;

1: On back-door (Sent, sid,  $S, R, |m|$ ) from  $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ :
2:   sample  $(\text{vk}_S, \text{sk}_S) \leftarrow \text{Sig.KGen}, (\text{pk}_R, \text{sk}_R) \leftarrow \text{KEM.Gen}$ ; store  $(\text{vk}_S, \text{pk}_R)$ 
3:   Set  $\text{m}_S \leftarrow \text{vk}_S$  and  $\text{m}_R \leftarrow \text{pk}_R$  and emulate  $(\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon))$  with  $(\text{m}_S, \text{m}_R, \text{sid})$ .
4: On the first detected misbinding attempt in the emulated cross-auth:
5:   (Detection e.g.: the adversary steers the run so that the key-path seen by  $S/R$ 
   is inconsistent with  $(\text{vk}_S, \text{pk}_R)$ .)
6: if  $\neg \text{attempted}$  then
7:    $(\text{MisbindResult}, \text{sid}, b) \leftarrow \mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon).\text{MisbindTry}(\text{sid})$    (locks the Bernoulli
   bit)
8:   set  $\text{attempted} \leftarrow \text{true}, \text{forged} \leftarrow b$ 
9:   Emulate  $\mathcal{F}_{\text{AUTH}}^{S \leftrightarrow R}(\varepsilon)$  one-shot leakage consistently with  $b$ 
10: end if
11:   (Ignore further misbind detections for this sid.)
12: On back-door (Reveal, sid,  $m$ ) from  $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ :
13:   store  $m$  (the attacker learns the original message in the ideal; confidentiality
   is intentionally void in this branch).
14: On emulated cross-auth (ok, sid) (scheduler's choice):
15: if  $\text{attempted} = \text{false}$  then

```

Final Simulator $\mathcal{S}$ (cont.)

```

16:   output (XAuth, sid, S, vkS) to R and (XAuth, sid, R, pkR) to S
17: else if forged = true then
18:   (deliver substituted keys consistently, per the emulation policy)
19:   output (XAuth, sid, S, vk'S) to R and (XAuth, sid, R, pk'R) to S (one valid
    choice; ensures R keeps pkR)
20: else
21:   output nothing (attempted and failed)
22: end if
23: Sender-side placement:
24: if forged = true and  $m \neq \perp$  and  $\text{pk}'_R \neq \perp$  and  $\neg \text{origEmitted}$  then
25:    $(K', c_{\text{KEM}}^{R'}) \leftarrow \text{KEM.Encap}(\text{pk}'_R)$ 
26:    $c_{\text{DEM}}^{R'} \leftarrow \text{DEM.ENC}(K', m)$ 
27:    $\sigma^S \leftarrow \text{Sig.Sign}_{\text{sk}_S}(\text{ctx}, c_{\text{KEM}}^{R'}, c_{\text{DEM}}^{R'})$ 
28:   place  $(\text{ctx}, c_{\text{KEM}}^{R'}, c_{\text{DEM}}^{R'}, \sigma^S)$  on the  $S \rightarrow R$  wire
29:   set origEmitted  $\leftarrow$  true
30: end if
31: On a data tuple  $(\text{ctx}, c_{\text{KEM}}, c_{\text{DEM}}, \sigma)$  heading to R:
32:   let  $b_{\text{Vfy}} \leftarrow \text{Sig.Vfy}_{\text{vk}}(\sigma; \text{ctx}, c_{\text{KEM}}, c_{\text{DEM}})$  (verifies under vk delivered to R)
33:   if  $b_{\text{Vfy}} = 1$  then
34:      $K \leftarrow \text{KEM.Decap}_{\text{sk}_R}(c_{\text{KEM}})$ ; if decap fails, treat as reject (abort then)
35:      $m' \leftarrow \text{DEM.DEC}_K(c_{\text{DEM}})$ ; if dec fails, treat as reject (abort then)
36:     if forged = true then
37:       (MitM success path) call  $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon).\text{SetMitmPayload}(\text{sid}, m')$ 
38:     end if
39:   end if
40: On back-door (ok, sid) from A:
41:   forward (ok, sid) to  $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$  unconditionally.
42:   (By  $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}(\varepsilon)$ 's tri-state:
43:   no attempt  $\Rightarrow$  deliver  $m$ ; attempt  $\wedge$  success  $\wedge$  payload set  $\Rightarrow$  deliver  $m'$  (and
    Reveal already sent); attempt  $\wedge$  fail  $\Rightarrow$  no delivery.)

```

E A Note on QR-codes

With smartphones and desktops we have a natural authenticated channel already available out-of-the-box, which might be from the first sight very powerful. A QR code offers an authenticated visual channel (the user sees which screen it came from) but not a confidential one (any onlooker can photograph it), and its payload is tiny. Therefore, transmitting secret information such as passwords or secret keys is inherently insecure. Transmitting public information such as a public key or certificate verbatim is possible, but solves only one part of the problem. The reason is that the canonical construction [Can20, BGMQ⁺22, SBB⁺23] of secure message transfer not only requires that the public key of a public-key encryption scheme is authenticated but also that the subsequent ciphertext is sent over an authenticated channel, which the unidirectional QR code cannot provide.

Verification Keys over QR-Codes Another option for transmitting public material over the QR channel to kickstart the authentication of arbitrary data chunks is to use the verification key of a digital-signature scheme. Roughly, the protocol proceeds as follows: the desktop generates an ephemeral key pair for a digital signature and embeds the verification key in the QR code. The smartphone scans the code (thereby receiving it over the authenticated channel) while, in parallel, the desktop sends its signed public key over the insecure network; the smartphone then verifies that signature.

Next, the smartphone generates a fresh, random nonce and sends it—encrypted under the desktop’s public key—back over the network, together with an ephemeral public key of its own. The desktop decrypts the message, extracts the nonce, and displays it. The user compares this displayed nonce with the one shown on the smartphone screen; a match confirms that both devices share authentic key material.

Finally, the desktop encrypts the file under the smartphone’s ephemeral public key and signs it with its signing key. The smartphone decrypts the ciphertext and verifies the signature.

QR-Codes are not Necessary The security of the previous protocol is evident for the most part, as it reduces to the authenticity of the QR-code channel, which is a powerful assumption/setup. Basically, we kickstart $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ from an existing unidirectional authenticated channel $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}$. Unfortunately, the QR-code channel is also a crucial bottleneck. This is not primarily due to its limited bandwidth (though that can still be a nuisance). The bottleneck becomes clear when generalizing to arbitrary device-to-device transmissions, such as desktop-to-desktop, that lack the means to scan QR codes. Other means of providing an authenticated channel for transferring the verification key, without prior setup and in a user-friendly way, are scarce.

One could, for example, use copy-and-paste to encode the verification key into a URL or file and transmit it to the second device over an authenticated channel, e.g. by sharing it via Signal, WhatsApp, or similar, but the payload may still be large and this initial data transfer might turn out a nuisance, and we seek a solution whose efficiency does not depend on key size. Moreover, any method that purports to bootstrap an authenticated channel $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}$ “out of nowhere” solely to deliver the verification key—thereby realizing $\mathcal{F}_{\text{AUTH}}^{S \rightarrow R}$ itself and then kickstarting $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ —is tautological. Consequently, such approaches could just as well bootstrap $\mathcal{F}_{\text{SMT}}^{S \rightarrow R}$ directly, which is exactly what we achieve in this work.

F Appendix: Removing the Random Oracle via Coin-Flip & Bind

Do not hash the transcript for the SAS. Jointly sample a simulatable public $c \in \{0, 1\}^t$ via a coin-flip, then bind c to this run with standard primitives. The UX is unchanged (users compare a t -bit string), and the simulator gains a programmable handle without a random oracle. A simple commit—then—XOR

coin toss in the style of Blum [Blu83] becomes UC in the CRS-hybrid using UC commitments [CF01, CLOS02].

Ideal Functionality $\mathcal{F}_{\text{coin}}^{S \leftrightarrow R}$

Parties: S , R , adversary $\mathcal{A}$.

On input (Flip, sid) from both S and R (order arbitrary), sample $c \leftarrow U(\{0, 1\}^t)$ and send (Coin, sid, c) to S , R , and $\mathcal{A}$. Ignore all other inputs.

Protocol $\Pi_{\text{SAS}}^{\text{ef}}$ (RO-free)

Public primitives:

- Commitment $\text{Com}(M; \rho)$ with opening $\text{Open}(C, M, \rho)$; Com is *hiding*, *binding*, and *non-malleable* in the UC sense (e.g., CRS-based UC commitments [CF01, Lin11]).
- $\text{MAC}_K(\cdot)$ (UF-CMA).

Round 1 (dual commits).

- S : sample $r_S, \rho_S \leftarrow U(\{0, 1\}^\kappa)$; send $C_S := \text{Com}((m_S, r_S); \rho_S)$ to R .
- R : sample $r_R, \rho_R \leftarrow U(\{0, 1\}^\kappa)$; send $C_R := \text{Com}((m_R, r_R); \rho_R)$ to S .

Round 2 (coin). Both invoke $\mathcal{F}_{\text{coin}}^{S \leftrightarrow R}$ and obtain public $c \in \{0, 1\}^t$ to display as the SAS (coin-toss UC via $\text{commit} \oplus$ in the CRS-hybrid [CLOS02, CF01]).

Round 3 (reveal headers). $S \rightarrow R$: (m_S, r_S) ; $R \rightarrow S$: (m_R, r_R) .

Round 4 (bind c with pre-open secret MACs).

$$\begin{aligned}\tau_S &:= \text{MAC}_{\rho_S}(\text{"bind"} \parallel \text{sid} \parallel m_S \parallel m_R \parallel r_S \parallel r_R \parallel c), \\ \tau_R &:= \text{MAC}_{\rho_R}(\text{"bind" } \parallel \text{sid} \parallel m_S \parallel m_R \parallel r_S \parallel r_R \parallel c).\end{aligned}$$

$S \rightarrow R$: τ_S ; $R \rightarrow S$: τ_R . (*Unforgeable at this point since ρ_S, ρ_R remain secret.*)

Round 5 (open and verify).

- $S \rightarrow R$: (m_S, r_S, ρ_S) with check $\text{Open}(C_S, (m_S, r_S), \rho_S) = \text{true}$;
- $R \rightarrow S$: (m_R, r_R, ρ_R) with check $\text{Open}(C_R, (m_R, r_R), \rho_R) = \text{true}$;

both verify τ_S under ρ_S and τ_R under ρ_R ; abort on any failure.

Round 6 (human compare). Display c to S and R ; compare via $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$; proceed only on match.

Assumptions. (i) $\mathcal{F}_{\text{coin}}^{S \leftrightarrow R}$ (or a UC realization) outputs uniform c [Blu83, CLOS02]; (ii) Com is hiding, binding, and *non-malleable* in UC (e.g., CRS-based constructions [CF01, Lin11]); (iii) MAC is UF-CMA; (iv) $\mathcal{F}_{\text{OOB}}^{\text{SAS}}$ faithfully delivers the t -bit comparison. The public session label sid is included for domain separation but provides no secrecy.

Why use ρ as the MAC key? ρ is fresh, uniform, and secret until the opening; binding of the commitment enforces a unique opening (in standard UC commitments, multiple openings w.h.p. would violate binding). Using ρ as the pre-open MAC key gives each party a one-shot secret to authenticate the binding of (m_S, m_R, r_S, r_R, c) *before* the opening values are revealed, and it avoids introducing extra key material. We leave the full proof for future work.